

PYTHON

- Python is a popular general-purpose programming language.
- Python is a simple, general purpose, high level, powerful and very popular programming language.
- **Guido Van Rossum** is known as the founder of Python programming. (1991)
- Python is simple and easy to learn and provides lots of high-level data structures.
- Python programming language (latest Python 3) is being used in web development, Machine Learning applications, along with all cutting-edge technology in Software Industry.
- Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc.

The biggest strength of Python is huge collection of standard library which can be used for the following:

Machine Learning

Machine learning (ML) is a branch of artificial intelligence (AI) and computer science that focuses on the using data and algorithms to enable AI to imitate the way that humans learn, gradually improving its accuracy, that focuses on analyzing and interpreting patterns and structures in data to enable learning, reasoning, and decision making outside of human interaction.

Artificial Intelligence

Artificial intelligence is the science of making machines that can think like humans. It can do things that are considered "smart." AI technology can process large amounts of data in ways, unlike humans. The goal for AI is to be able to do things such as recognize patterns, make decisions, and judge like humans.

Data Science

Data science is the study of data to extract meaningful insights for business. It is a multi-disciplinary approach that combines principles and practices from the fields of mathematics, statistics, artificial intelligence, and computer engineering to analyze large amounts of data.

GUI Applications (like [Kivy](#), Tkinter, PyQt etc.)

Graphical User Interface (GUI), a computer program that enables a person to communicate with a computer through the use of symbols, visual metaphors, and pointing devices.

Web frameworks like [Django](#) (used by YouTube, Instagram, Dropbox)

In computer systems, a framework is often a layered structure indicating what kind of programs can or should be built and how they would interrelate. Some computer system frameworks also include actual programs, specify programming interfaces, or offer programming tools for using the frameworks.

Image processing (like [OpenCV](#), Pillow)

Image processing is done to enhance an existing image or to sift out important information from it. This is important in several Deep Learning-based Computer Vision applications, where such preprocessing can dramatically boost the performance of a model.

Web scraping (like Scrapy, BeautifulSoup, Selenium)

Web scraping is an automatic method to obtain large amounts of data from websites.

Test frameworks

Text processing is the automated process of analyzing and sorting unstructured-text data to gain valuable insights.

Multimedia

Multimedia is a type of medium that allows information to be easily transferred from one location to another.

Scientific computing ig

Scientific Computing is nowadays the “third pillar of science”, standing right next to theoretical analysis and experiments for scientific discovery.

Computer Vision or Image Processing Applications

Image processing is the process of transforming an image into a digital form and performing certain operations to get some useful information from it. The image processing system usually treats all images as 2D signals when applying certain predetermined signal processing methods.

Text processing and many more..

Text processing is the automated process of analyzing and sorting unstructured text data to gain valuable insights.

Desktop & Mobile Applications

Desktop applications are software programs specifically designed to run on desktop or laptop computers.

- A desktop application is a dedicated software program designed to run on a standalone computer, enabling end-users to execute specific tasks.
- A mobile app (or mobile application) is a software application developed specifically for use on small, wireless computing devices, such as smartphones and tablets, rather than desktop or laptop computers.

Why Learn Python?

- Easy to use and Learn
- Expressive Language
- Interpreted Language
- Object-Oriented Language
- Open-Source Language
- Extensible
- Integrated
- Embeddable
- Dynamic Memory Allocation

Java vs Python Program

Unlike the other programming languages, Python provides the facility to execute the code using few lines. **For example** - Suppose we want to print the "**Hello World**" program in Java; it will take three lines to print it.

Java Program

1. **public class** HelloWorld
2. {
3. **public static void** main(String[] args)
4. {
5. System.out.println("Hello World");
6. }
7. }

Python Program

On the other hand, we can do this using one statement in Python.

- **print("Hello World")**

Both programs will print the same result, but it takes only one statement without using a semicolon or curly braces in Python.

Python print() Function

Here, The **print()** function displays the given object to the standard output device (screen) or to the text stream file.

Unlike the other programming languages, Python **print()** function is most unique and versatile function.

The syntax of **print()** function is given below.

print(*objects, sep=" ", end='\n')

Let's explain its parameters one by one.

- **objects** - An object is nothing but a statement that to be printed. The * sign represents that there can be multiple statements.
- **sep** - The **sep** parameter separates the print values. Default values is " ".
- **end** - The **end** is printed at last in the statement. Default values is "\n".

Sample Program

1. **a = 10**

2. **print("a =", a) # Two objects are passed in print() function**

O/p :- a = 10

=====

Identifiers

- Python Identifier is the name we give to a variable, function, class, module or other object to identify it during the execution of the program.
- That means whenever we want to give an entity a name, that's called identifier.
- Sometimes variable and identifier are often misunderstood as same but they are not.

Python Variable

- Variable is a name that is used to refer to memory location. Python variable is also known as an identifier and used to hold value.
- Python Variable is a container that stores values.
- Python is “dynamically typed” language. Unlike other programming languages, We do not need to declare variables before using them or declare their type.
- Python has no command for declaring a variable.
- A variable is created the moment we first assign a value to it. This means that when you create a variable you reserve some space in the memory.
- The equal sign (=) is used to assign values to variables.

Example

```
x = 4      # x is of type int
x = "Sally" # x is now of type str
print(x)
```

Note :

- An Example of a Variable in Python is a representational name that serves as a pointer to an object. Once an object is assigned to a variable, it can be referred to by that name.
- The value stored in a variable can be changed during program execution.
- A Variables in Python is only a name given to a memory location, all the operations done on the variable effects that memory location.

Rules for Python variables

- A Python variable name must start with a letter or the underscore character.
- A Python variable name cannot start with a number.
- A Python variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _).
- Variable in Python names are case-sensitive (name, Name, and NAME are three different variables).
- The reserved words (keywords) in Python cannot be used to name the variable in Python.

valid variable name

```
thops = 1
```

```
Thops = 2
```

```
Th_o_ps = 5
```

```
_thops = 6
```

```
thops_ = 7
```

```
_THOPS_ = 8
```

```
print(thops, Thops, Th_o_ps)
```

```
print(_thops, thops_, _THOPS_)
```

Output:

```
1 2 5
```

```
6 7 8
```

Keywords

- Keywords are reserved words that have special meaning to the Python Interpreter.
- Keywords can not be used as a variable name, function name, or any other identifier.

- Keywords reserved with defined meanings and functions that we can only apply for those functions.
- You'll never need to import any keyword into your program because they're permanently present.

Here's a list of all keywords in Python Programming

Keywords in Python programming language				
False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

The above keywords may get altered in different versions of Python. Some extra might get added or some might be removed. You can always get the list of keywords in your current version by typing the following in the prompt.

```
>>> import keyword
```

```
>>> print(keyword.kwlist)
```

Output :

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class',
'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if',
'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while',
'with', 'yield']
```

An **EXPRESSION** is a combination of operators and operands that is interpreted to produce some other value.

OPERATORS: These are the special symbols. Eg- + , * , /, etc.

OPERAND: It is the value on which the operator is applied.

Example

x = 15 + 1.3

here, = + are operators and x, 15 & 1.3 are operands

Operators

The operator is a symbol that performs a certain operation between two operands.

Types of Python Operators

Here's a list of different types of Python operators that we will learn in this tutorial.

1. Arithmetic operators
2. Assignment Operators
3. Comparison Operators
4. Logical Operators
5. Bitwise Operators
6. Special Operators

Arithmetic Operators

Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication, etc.

Operator	Operation	Example
+	Addition	5 + 2 = 7
-	Subtraction	4 - 2 = 2
*	Multiplication	2 * 3 = 6
/	Division	4 / 2 = 2
//	Floor Division	10 // 3 = 3
%	Modulo	5 % 2 = 1
**	Power	4 ** 2 = 16

Sample Program

```

1 a = 10
2 b = 3
3
4 # addition
5 print ('Sum: ', a + b)
6
7 # subtraction
8 print ('Subtraction: ', a - b)
9
10 # multiplication
11 print ('Multiplication: ', a * b)
12
13 # division
14 print ('Division: ', a / b)
15
16 # floor division
17 print ('Floor Division: ', a // b)
18
19 # modulo
20 print ('Modulo: ', a % b)
21
22 # a to the power b
23 print ('Power: ', a ** b)
24

```

Sum: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333335
Floor Division: 3
Modulo: 1
Power: 1000
>

Comparison Operators

Comparison operators compare two values/variables and return a boolean result: True or False.

Operator	Meaning	Example
<code>==</code>	Is Equal To	<code>3 == 5</code> gives us False
<code>!=</code>	Not Equal To	<code>3 != 5</code> gives us True
<code>></code>	Greater Than	<code>3 > 5</code> gives us False
<code><</code>	Less Than	<code>3 < 5</code> gives us True
<code>>=</code>	Greater Than or Equal To	<code>3 >= 5</code> give us False
<code><=</code>	Less Than or Equal To	<code>3 <= 5</code> gives us True

Sample Program

```

1  a = 5
2  b = 2
3
4  # equal to operator
5  print('a == b =', a == b)
6
7  # not equal to operator
8  print('a != b =', a != b)
9
10 # greater than operator
11 print('a > b =', a > b)
12
13 # less than operator
14 print('a < b =', a < b)
15
16 # greater than or equal to operator
17 print('a >= b =', a >= b)
18
19 # less than or equal to operator
20 print('a <= b =', a <= b)
21

```

```

a == b = False
a != b = True
a > b = True
a < b = False
a >= b = True
a <= b = False
> |

```

Logical Operators

Logical operators are used to check whether an expression is True or False. They are used in decision-making.

Operator	Example	Meaning
<code>and</code>	<code>a and b</code>	Logical AND: <code>True</code> only if both the operands are <code>True</code>
<code>or</code>	<code>a or b</code>	Logical OR: <code>True</code> if at least one of the operands is <code>True</code>
<code>not</code>	<code>not a</code>	Logical NOT: <code>True</code> if the operand is <code>False</code> and vice-versa.

Sample Program

1	# logical AND		True
2	print(True and True)	# True	False
3	print(True and False)	# False	True
4			False
5	# logical OR		>
6	print(True or False)	# True	
7			
8	# logical NOT		
9	print(not True)	# False	
10			

Assignment Operators

Assignment operators are used to assign values to variables.

```
# assign 5 to x
var x = 5
```

Here, `=` is an assignment operator that assigns `5` to `x`.

Operator	Name	Example
=	Assignment Operator	a = 7
+=	Addition Assignment	a += 1 # a = a + 1
-=	Subtraction Assignment	a -= 3 # a = a - 3
*=	Multiplication Assignment	a *= 4 # a = a * 4
/=	Division Assignment	a /= 3 # a = a / 3
%=	Remainder Assignment	a %= 10 # a = a % 10
**=	Exponent Assignment	a **= 10 # a = a ** 10

Sample Program

1	a = 10	15
2	b = 5	10
3		50
4	# assign the sum of a and b to a	>
5	a += b # a = a + b	
6	print(a)	
7		
8	# assign the diff of a and b to a	
9	a -= b # a = a - b	
10	print(a)	
11		
12	# assign the product of a and b to a	
13	a *= b # a = a * b	
14	print(a)	
15		

Special Operators

Python language offers some special types of operators like the **identity** operator and the **membership** operator. They are described below with examples.

Identity operators

In Python, `is` and `is not` are used to check if two values are located on the same part of the memory. Two variables that are equal does not imply that they are identical.

Operator	Meaning	Example
<code>is</code>	<code>True</code> if the operands are identical (refer to the same object)	<code>x is True</code>
<code>is not</code>	<code>True</code> if the operands are not identical (do not refer to the same object)	<code>x is not True</code>

1	<code>x1 = 100</code>	False
2	<code>y1 = 100</code>	True
3	<code>x2 = 'Python'</code>	False
4	<code>y2 = 'Python'</code>	>
5	<code>x3 = [1,2,3]</code>	
6	<code>y3 = [1,2,3]</code>	
7		
8	<code>print(x1 is not y1)</code> # prints False	
9	<code>print(x2 is y2)</code> # prints True	
10	<code>print(x3 is y3)</code> # prints False	

Bitwise Operators

Bitwise operators act on operands as if they were strings of binary digits. They operate bit by bit, hence the name.

For example, **2** is `10` in binary and **7** is `111`.

In the table below: Let `x` = 10 (`0000 1010` in binary) and `y` = 4 (`0000 0100` in binary)

Operator	Meaning	Example
<code>&</code>	Bitwise AND	<code>x & y = 0</code> (<code>0000 0000</code>)
<code> </code>	Bitwise OR	<code>x y = 14</code> (<code>0000 1110</code>)
<code>~</code>	Bitwise NOT	<code>~x = -11</code> (<code>1111 0101</code>)
<code>^</code>	Bitwise XOR	<code>x ^ y = 14</code> (<code>0000 1110</code>)
<code>>></code>	Bitwise right shift	<code>x >> 2 = 2</code> (<code>0000 0010</code>)
<code><<</code>	Bitwise left shift	<code>x << 2 = 40</code> (<code>0010 1000</code>)

Datatypes

Datatype is a classification that specifies the Python interpreter how the programmer intends to use the data (what type of value a variable has and what type of operations can be applied without causing an error).

Data types specify the different sizes and values that can be stored in the variable.

- Python is “dynamically typed” language. Unlike other programming languages, We do not need to declare variables before using them or declare their type.
- Python has no command for declaring a variable.
- A variable is created the moment we first assign a value to it. This means that when you create a variable you reserve some space in the memory.
- The equal sign (=) is used to assign values to variables.

Variables can store data of different types, and different types can do different things.

Python has the following data types built-in by default, in these categories:

Text Type:	<code>str</code>
Numeric Types:	<code>int</code> , <code>float</code> , <code>complex</code>
Sequence Types:	<code>list</code> , <code>tuple</code> , <code>range</code>
Mapping Type:	<code>dict</code>
Set Types:	<code>set</code> , <code>frozenset</code>
Boolean Type:	<code>bool</code>
Binary Types:	<code>bytes</code> , <code>bytearray</code> , <code>memoryview</code>
None Type:	<code>NoneType</code>

Type Casting

The conversion of one data type into the other data type is known as type casting in python or type conversion in python.

There may be times when you want to specify a type on to a variable. This can be done with casting. Python is an object-orientated language, and as such it uses classes to define data types, including its primitive types.

Python supports a wide variety of functions or methods like: `int()`, `float()`, `str()`, `ord()`, `hex()`, `oct()`, `tuple()`, `set()`, `list()`, `dict()`, etc. for the type casting in python.

There are two types of type conversion in Python.

- Implicit Conversion - automatic type conversion
- Explicit Conversion - manual type conversion

Implicit Type Conversion

Python automatically converts one data type to another. This is known as implicit type conversion.

Example 1: Converting integer to float

Let's see an example where Python promotes the conversion of the lower data type (integer) to the higher data type (float) to avoid data loss.

<pre>1 integer_number = 123 2 float_number = 1.23 3 4 new_number = integer_number + float_number 5 6 # display new value and resulting data type 7 print("Value:", new_number) 8 print("Data Type:", type(new_number))</pre>	<pre>Value: 124.23 Data Type: <class 'float'> > </pre>
--	--

In the above example, we have created two variables: `integer_number` and `float_number` of `int` and `float` type respectively.

Then we added these two variables and stored the result in `new_number`.

As we can see `new_number` has value 124.23 and is of the float data type.

It is because Python always converts smaller data types to larger data types to avoid the loss of data.

Note:

- We get `TypeError`, if we try to add `str` and `int`. For example, `'12' + 23`. Python is not able to use Implicit Conversion in such conditions.
- Python has a solution for these types of situations which is known as Explicit Conversion.

Explicit Type Conversion

In Explicit Type Conversion, users convert the data type of an object to required data type.

We use the built-in functions like `int()`, `float()`, `str()`, etc to perform explicit type conversion.

This type of conversion is also called typecasting because the user casts (changes) the data type of the objects.

Example 2: Addition of string and integer Using Explicit Conversion

<pre>1 num_str = '12' 2 num_int = 23 3 print("Datatype of num_str before Type Casting:",type(num_str)) 4 # explicit type conversion 5 num_str = int(num_str) 6 print("Data type of num_str after Type Casting:",type(num_str)) 7 num_sum = num_int + num_str 8 print("Sum:",num_sum) 9 print("Data type of num_sum:",type(num_sum))</pre>	<pre>Datatype of num_str before Type Casting: <class 'str'> Data type of num_str after Type Casting: <class 'int'> Sum: 35 Data type of num_sum: <class 'int'> > </pre>
---	---

In the above example, we have created two variables: `num_string` and `num_integer` with `str` and `int` type values respectively. Notice the code,

`num_string = int(num_string)`

Here, we have used `int()` to perform explicit type conversion of `num_string` to integer type.

After converting `num_string` to an integer value, Python is able to add these two variables.

Finally, we got the `num_sum` value i.e 35 and data type to be `int`.

In Type Casting, loss of data may occur as we enforce the object to a specific data type.

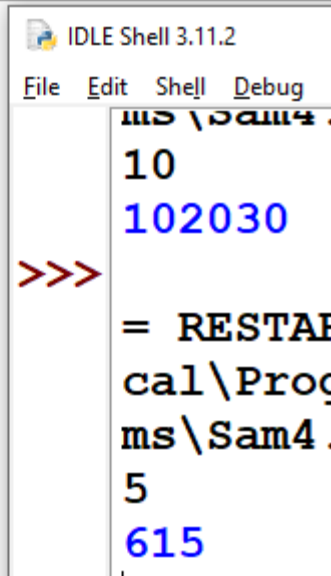
Programming Exercise

- Write a Python program that accepts an integer (`n`) and computes the value of `n+nn+nnn`.

Sample value of `n` is 5

Expected Result : 615

```
a = input()
b = a+a
c = a+a+a
sum = int(a)+int(b)+int(c)
print(sum)
```



The screenshot shows the IDLE Shell 3.11.2 interface. The left pane contains the Python code. The right pane shows the execution output: the input '10' is entered, and the output '102030' is printed. The prompt '>>>' is visible, and the file path 'ms\Sam4.' is partially visible at the top of the right pane.

- Write a Python program to calculate the difference between a given number and 17. If the number is greater than 17, return twice the absolute difference.

Sample value of n is 20

Expected Result : 6

Sample value of n is 5

Expected Result : 12

- Write a Python program to calculate the area of Square

Sample value of side is 10

Expected Result : The area of square with side 10 cm is 100 cm²

Sample value of n is 5

Expected Result : The area of square with side 6 cm is 36 cm²

- Write a Python program to calculate the area of Rectangle

Sample values of length & width are 20 & 10

Expected Result: The area of rectangle with length 20 cm and width 10 cm is 200 cm²

Sample values of length & width are 10 & 5

Expected Result: The area of rectangle with length 10 cm and width 5 cm is 50 cm²

- Write a Python program to find greatest among two numbers
- Write a Python program to find greatest among three numbers
- Write a Python program to find second greatest among three numbers

Control Statements

In Python, control flow starts from first line & eventually transfers line by line.

Control statements are used to control the flow of execution of the program. They allow you to execute different parts of the program based on certain conditions.

There are three main types of control statements in Python:

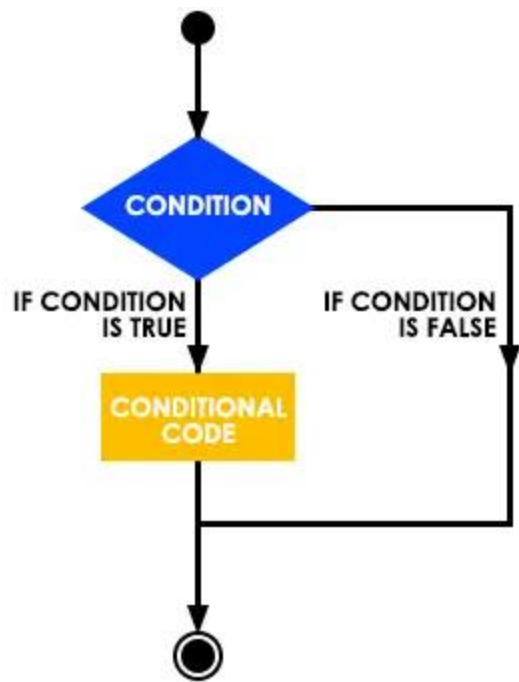
Decision Making statements: allow you to execute a block of code only if a certain condition is true.

The conditional statements (also known as decision control structures) such as if, if else, switch, etc.

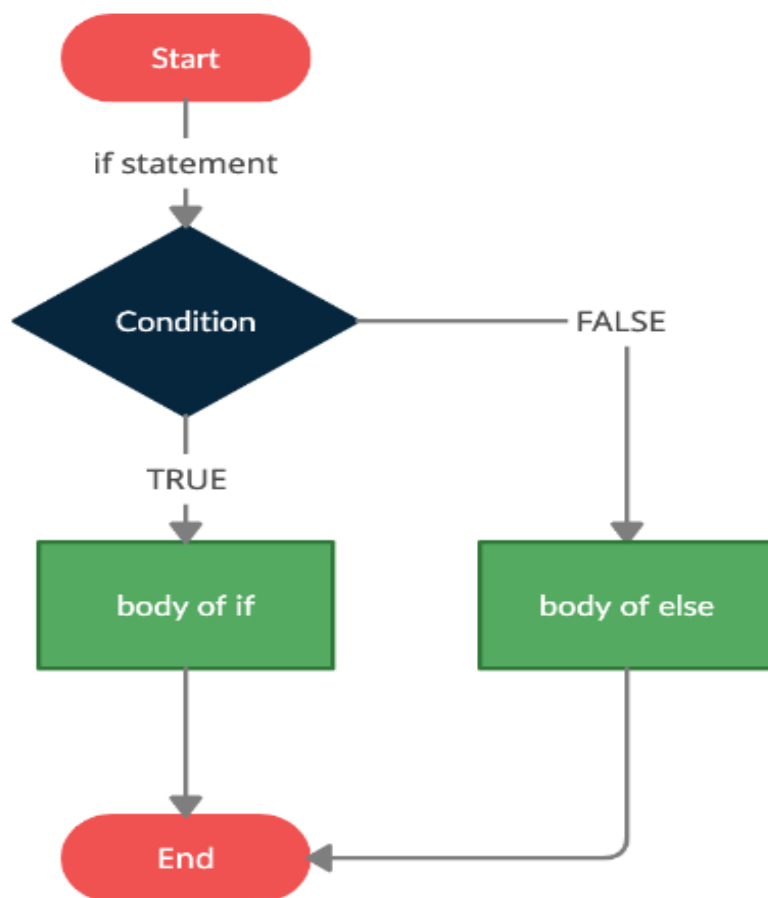
Decision-Making Statements are used to evaluate one or more conditions and make the decision whether to execute a set of statements or not.

The type of control flow statements is as follows:

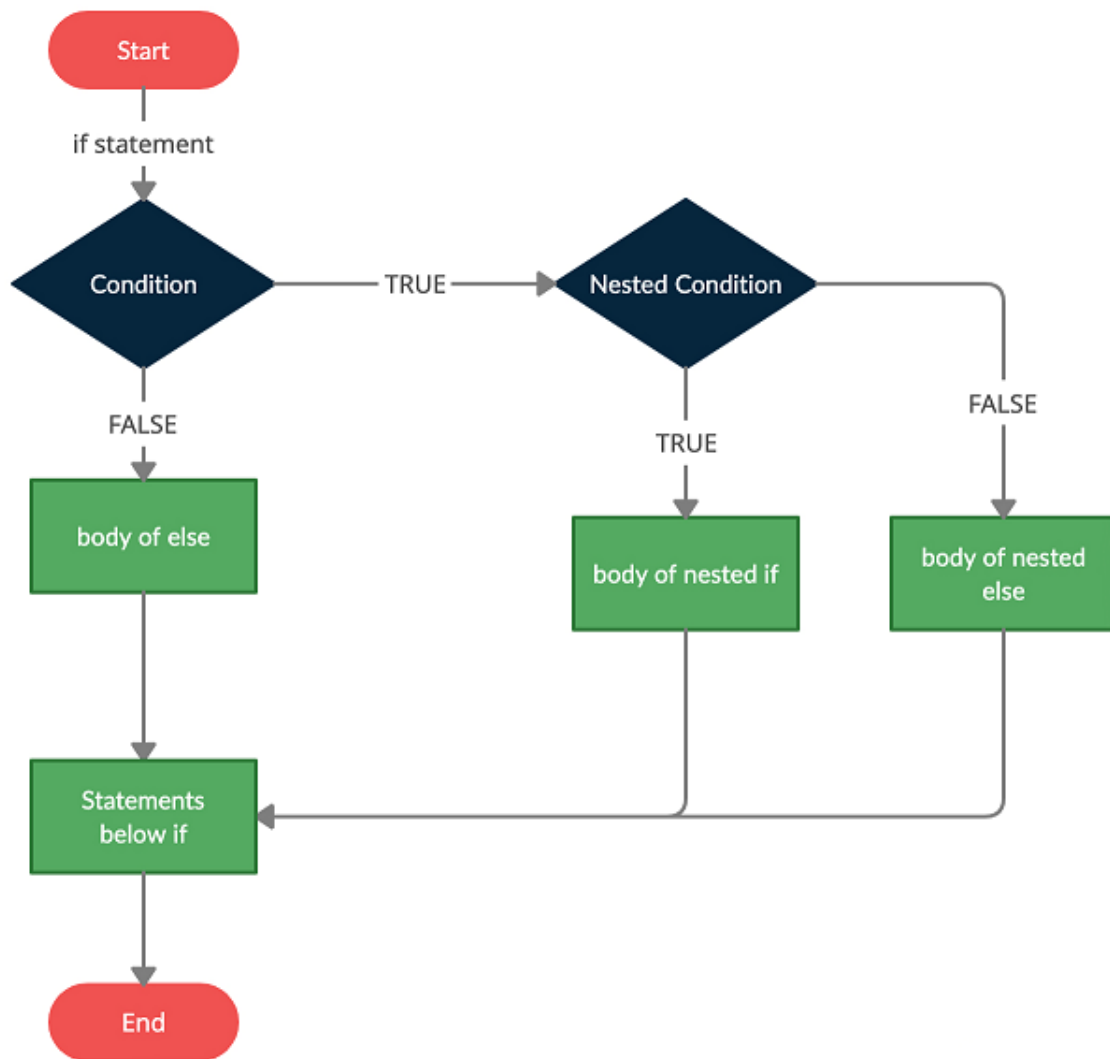
The **if** statement



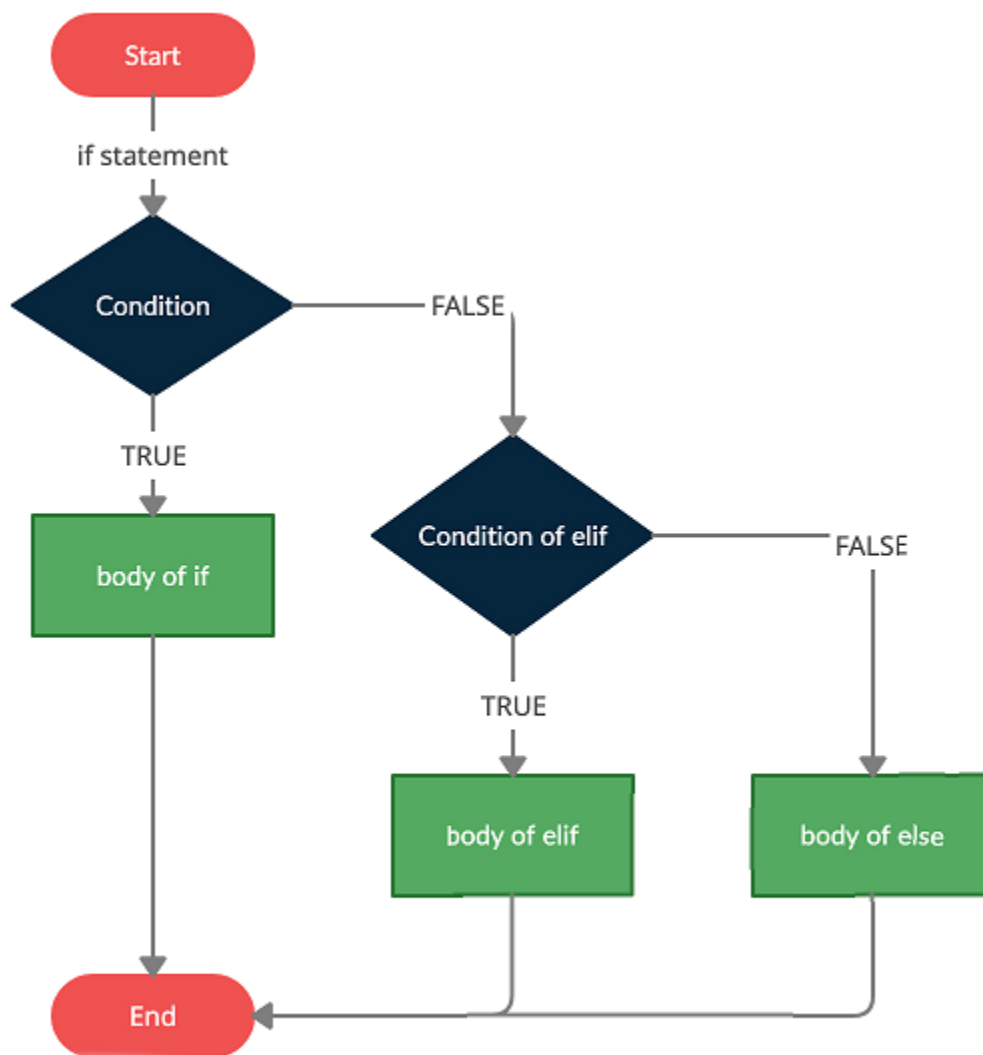
The **if-else** statement



The **nested-if** statement



The **if-elif-else** ladder

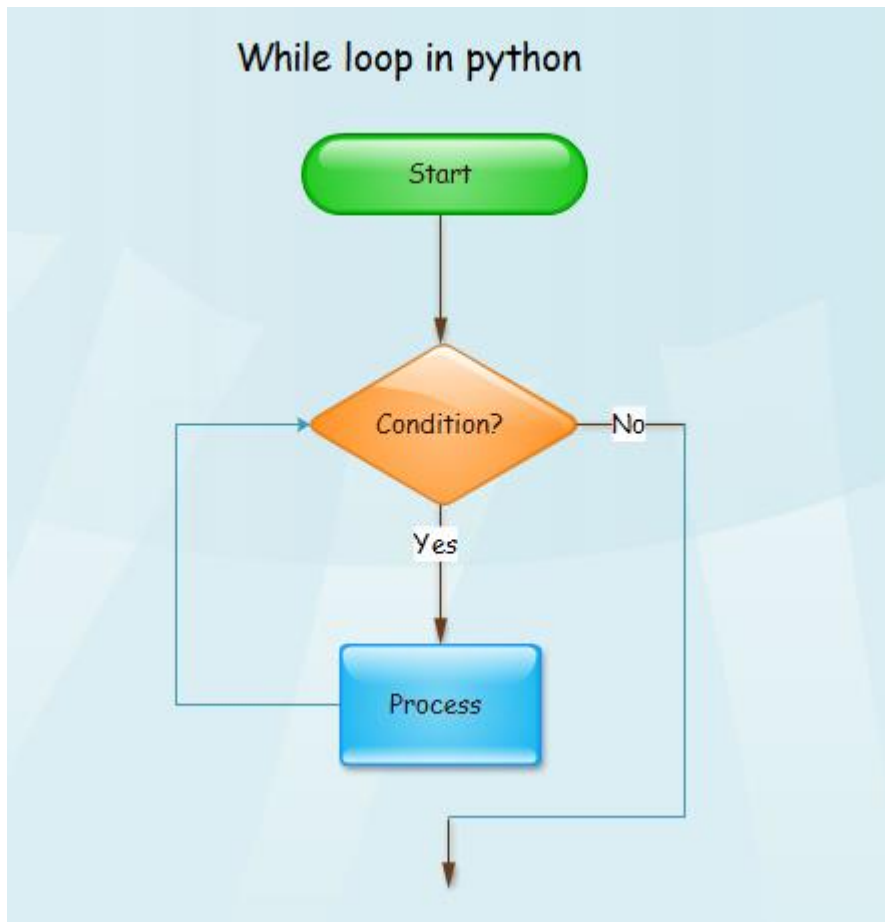


Loops: allow you to execute a block of code repeatedly, usually until a certain condition is met.

Loops are classified into two types

1. Conditional Loop

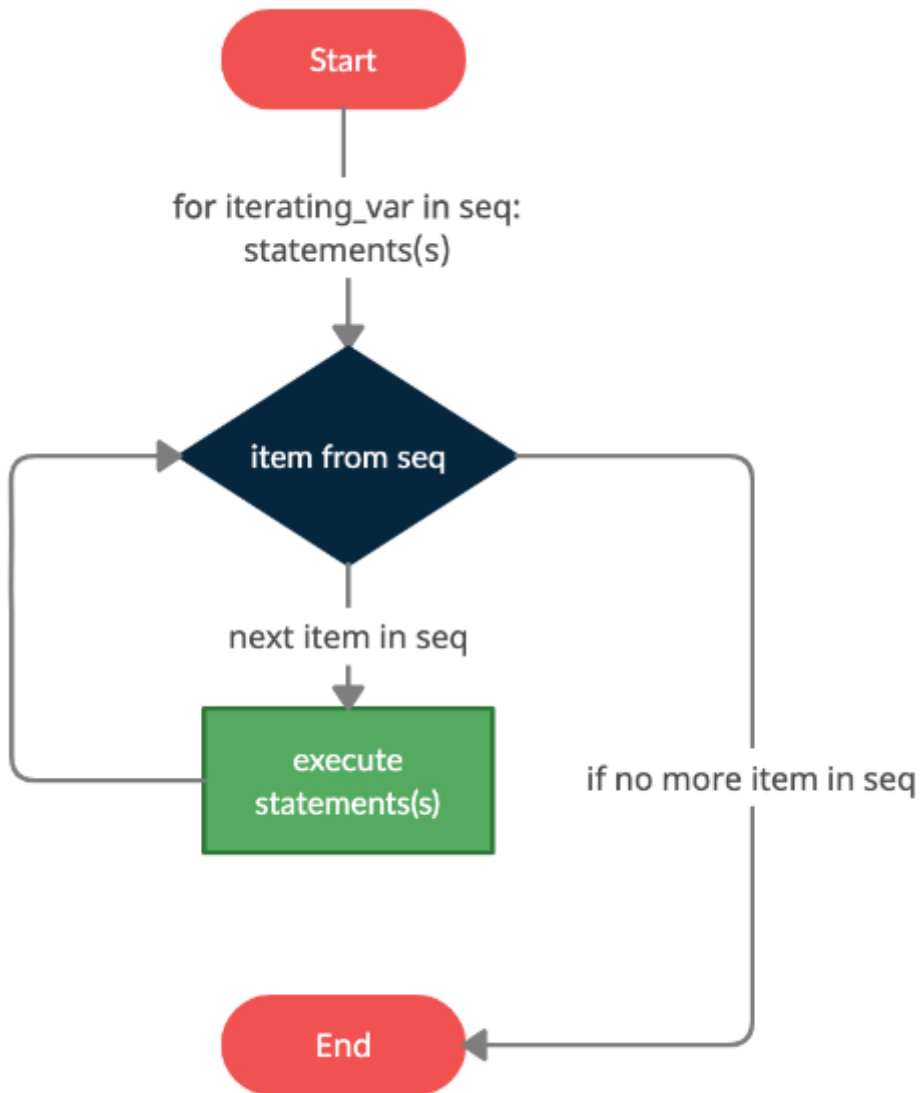
Loop is a mechanism that executes same set of statements until a specific condition is met.



2. *Traversal Loop*

Traversing in Data Structure means systematically visiting every element of given Sequence. Accessing an element of data structure means visiting every element at least once.

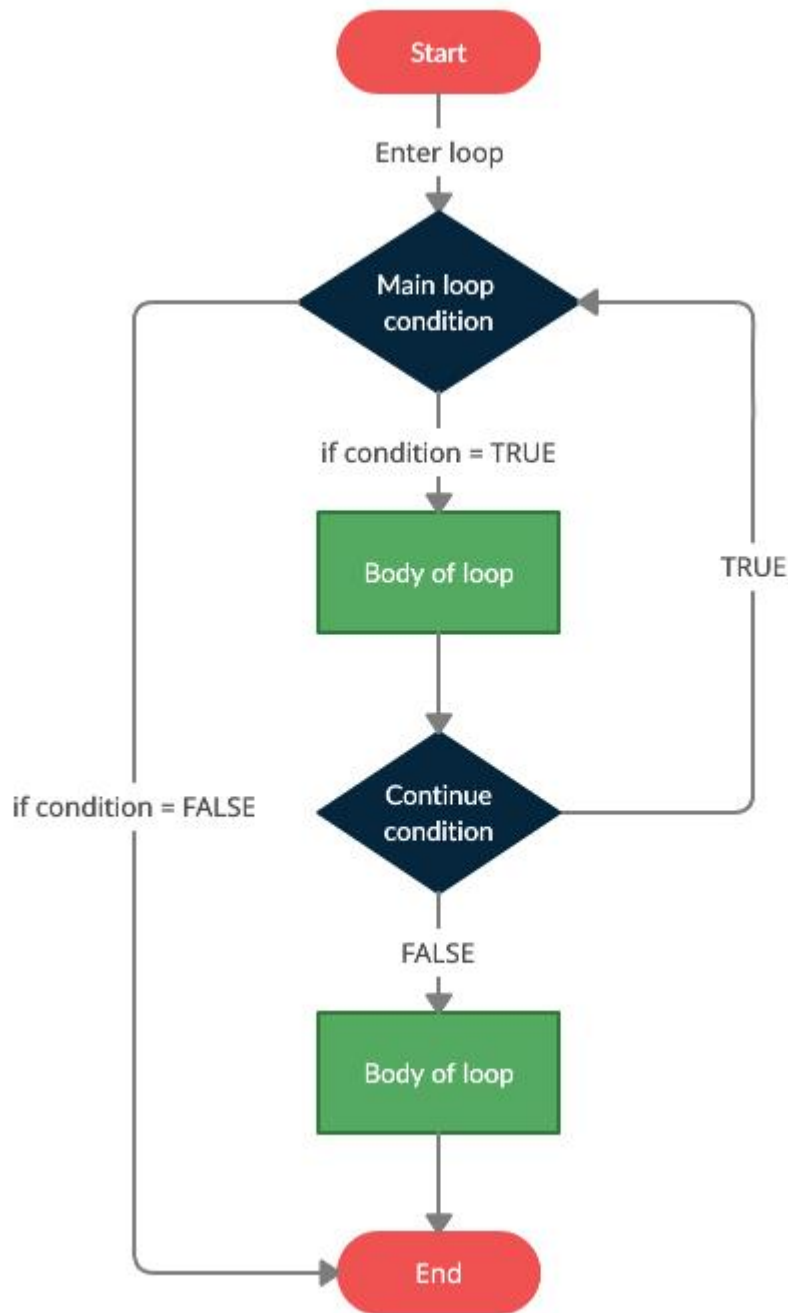
A "For" Loop is used to repeat a specific block of code a known number of times.



Jump statements: allow you to execute a different block of code based on the value of a variable.

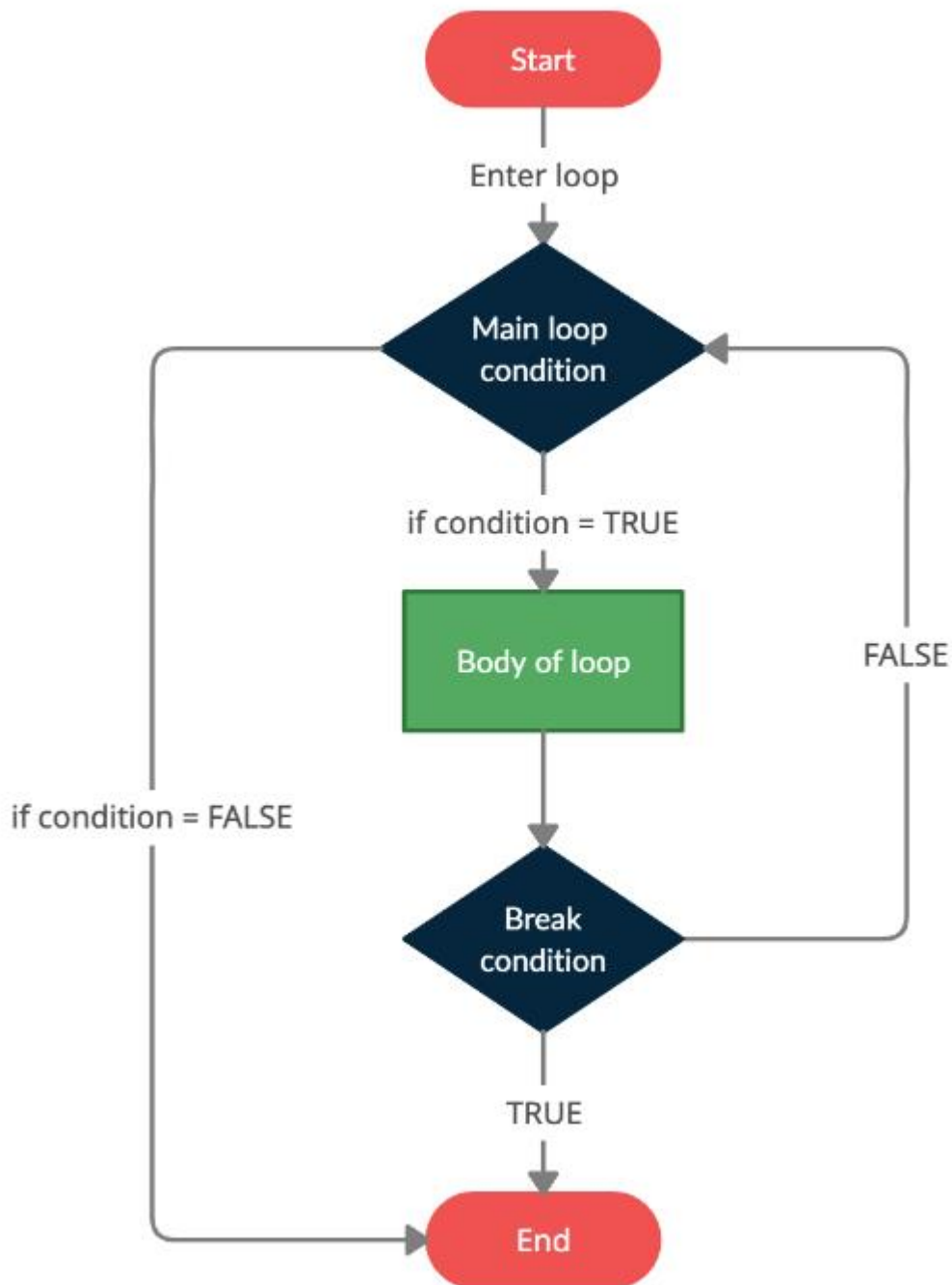
Continue

Python Continue statement is a loop control statement that forces to execute the next iteration of the loop while skipping the rest of the code inside the loop for the current iteration only, i.e. when the continue statement is executed in the loop, the code inside the loop following the continue statement will be skipped for the current iteration and the next iteration of the loop will begin.



Break

break statement is put inside the loop body (generally after if condition). It terminates the current loop, i.e., the loop in which it appears, and resumes execution at the next statement immediately after the end of that loop. If the break statement is inside a nested loop, the break will terminate the innermost loop.



range()

The Python `range()` function returns a sequence of numbers, in a given range. The most common use of it is to iterate sequences on a sequence of numbers using Python loops.

Syntax:

range(start, stop, step)

Parameter :=

start: [optional] start value of the sequence (default value : **0**)

stop: next value after the end value of the sequence

step: [optional] integer value, denoting the difference between any two numbers in the sequence (default value : **1**)

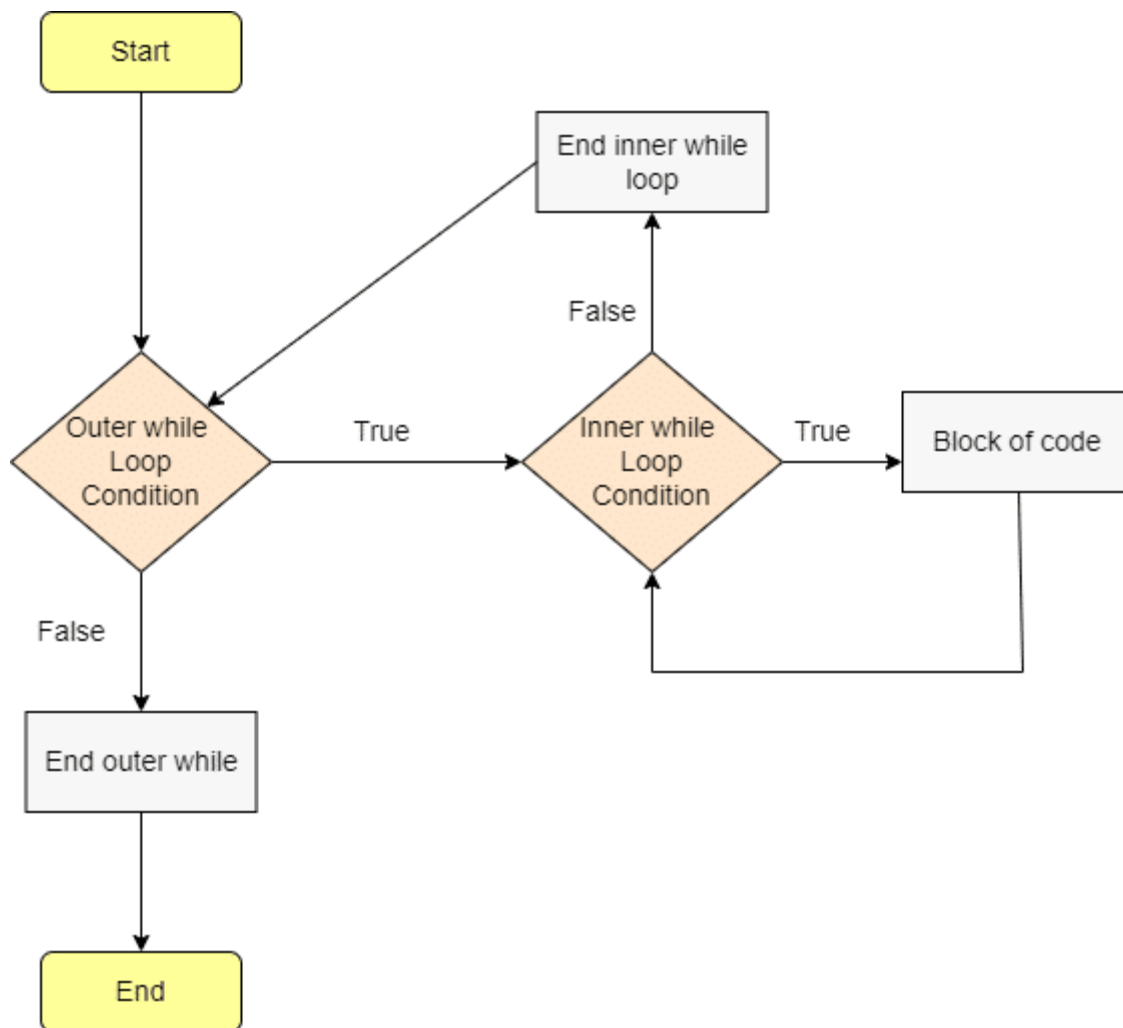
Return : Returns an object that represents a sequence of numbers

1 ▾ <code>for i in range(5):</code>	0 1 2 3 4
2 <code>print(i)</code>	
3	
4 ▾ <code>for i in range(1,5):</code>	1 2 3 4
5 <code>print(i)</code>	
6	
7 ▾ <code>for i in range(1,5,2):</code>	1 3
8 <code>print(i)</code>	>

Nested Loop

A nested loop is a loop inside another loop. Although all kinds of loops can be nested, the most common nested loop involves for loops. These loops are particularly useful when displaying multidimensional data. When using these loops, the first iteration of the first loop will initialize, followed by the second loop.

1 <code>n = int(input())</code>	5
2	1 2 3 4 5
3 ▾ <code>for i in range(1,n+1):</code>	1 2 3 4 5
4 ▾ <code>for j in range(1,n+1):</code>	1 2 3 4 5
5 <code>print(j, end=" ")</code>	1 2 3 4 5
6 <code>print()</code>	1 2 3 4 5



PATTERNS



pattern

/ˈpɑːtn/

noun

1. a repeated decorative design.
"a neat blue herringbone pattern"

Pattern programs are patterns/designs/symbols consisting of numbers, alphabets or symbols in a particular form. These kinds of pattern are solved using looping structures.

Pat - 6	Pat - 7	Pat - 10
1 2 3 4 5 5 4 3 2 1 1 2 3 4 5 5 4 3 2 1 1 2 3 4 5	1 1 1 1 1 5 4 3 2 1 3 3 3 3 3 5 4 3 2 1 5 5 5 5 5	5 4 3 2 1 10 9 8 7 6 15 14 13 12 11 20 19 18 17 16 25 24 23 22 21
Pat - 8	Pat - 9	Last
5 5 5 5 5 1 2 3 4 5 3 3 3 3 3 1 2 3 4 5 1 1 1 1 1	1 2 3 4 5 4 5 6 7 8 9 10 11 12 13 16 17 18 19 20 25 26 27 28 29 30	1 2 3 4 5 10 9 8 7 6 11 12 13 14 15 20 19 18 17 16 21 22 23 24 25

1 0 0 0 1 1 0 0 0 1 1 0 0 0 1 1 0 0 0 1 1 0 0 0 1 =====	1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 =====	1 1 1 1 1 1 0 0 0 1 1 0 1 0 1 1 0 0 0 1 1 1 1 1 1 (Only ODD input =====	1 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 1 =====
1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 =====	0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 (Only ODD input =====	25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 =====	0 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 0 =====
1 1 1 1 1 1 0 0 0 1 1 0 0 0 1 1 0 0 0 1 1 1 1 1 1	1 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 1 (Only ODD input	5 4 3 2 1 5 4 3 2 1 5 4 3 2 1 5 4 3 2 1 5 4 3 2 1 =====	1 0 0 0 1 0 1 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 1

5
44
333
2222
11111

Number pattern 23

1
12
123
1234
12345

Number pattern 24

12345
1234
123
12
1

Number pattern 25

1
21
321
4321
54321

Number pattern 26

54321
4321
321
21
1

Number pattern 27

5
54
543
5432
54321

Number pattern 28

54321
5432
543
54
5

Number pattern 29

5
45
345
2345
12345

Number pattern 30

12345
2345
345
45
5

Number pattern 31

1
23
345
4567
56789

Number pattern 32

56789
4567
345
23
1

Number pattern 33

Triangle 0,1 Easy Number Patterns

1
10
101
1010
10101

Number pattern 35

1
00
111
0000
11111

Number pattern 36

1
01
010
1010
10101

Number pattern 37

1
123
12345
1234567
123456789

Number pattern 39

1
24
135
2468
13579

Number pattern 40

1
131
13531
1357531
135797531

Number pattern 41

2
242
24642
2468642
2468108642

Number pattern 42

1
121
12321
1234321
123454321

Number pattern 43

```

1
32
4543
567654
67898765

```

Number pattern 44

```

1
2 3
4 5 6
7 8 9 10
11 12 13 14 15

```

Number pattern 45

```

1
21
123
4321
12345

```

Number pattern 46

```

1
2 6
3 7 10
4 8 11 13
5 9 12 14 15

```

Number pattern 49

```

1
2 4
7 11 16
22 29 37 46
56 67 79 92 106

```

Number pattern 50

```

1
3 2
4 5 6
10 9 8 7
11 12 13 14 15

```

Number pattern 51

```

1
22
333
2222
11111

```

Number pattern 52

N = 12345

```

12345
2345
345
45
5

```

Number pattern 54

Diamond Number Patterns

```

1
12
123
1234
12345
1234
123
12
1

```

Number pattern 55

```

1
123
12345
1234567
123456789
1234567
12345
123
1

```

Number pattern 56

```

5 5 5 5 5 5 5 5
5 4 4 4 4 4 4 5
5 4 3 3 3 3 3 5
5 4 3 2 2 2 3 5
5 4 3 2 1 2 3 5
5 4 3 2 2 2 3 5
5 4 3 3 3 3 3 5
5 4 4 4 4 4 4 5
5 5 5 5 5 5 5 5

```

Number pattern 18

```

5 5 5 5 5 5 5 5
5 4 4 4 4 4 4 5
5 4 3 3 3 3 3 5
5 4 3 2 2 2 3 5
5 4 3 2 1 2 3 5
5 4 3 2 2 2 3 5
5 4 3 3 3 3 3 5
5 4 4 4 4 4 4 5
5 5 5 5 5 5 5 5

```

Number pattern 18

```

1    2    3    4    5
16   17   18   19   6
15   24   25   20   7
14   23   22   21   8
13   12   11   10   9

```

Number pattern 19

```

1    2    3    4    5
16   17   18   19   6
15   24   25   20   7
14   23   22   21   8
13   12   11   10   9

```

Number pattern 19

Strings

A String is a data structure in Python that represents a sequence of characters surrounded by single quotes, double quotes, or triple quotes.

The computer does not understand the characters; internally, it stores manipulated character as the combination of the 0's and 1's.

Each character is encoded in the ASCII or Unicode character. So we can say that Python strings are also called the collection of Unicode characters.

Assigning a string to a variable is done with the variable name followed by an equal sign and the string:

You can display a string literal with the print() function

<pre>1 # create string type variables 2 name = "Python" 3 print(name) 4 message = "I love Python." 5 print(message)</pre>	<pre>Python I love Python. > </pre>
---	---

We can access the characters in a string in three ways.

1. **Indexing:** One way is to treat strings as a list and use index values. For example,

<pre>1 lang = 'python' 2 # access 1st index element 3 print(lang[1]) # "y"</pre>	<pre>y > </pre>
--	---------------------

2. **Negative Indexing:** Similar to a list, Python allows negative indexing for its strings. For example,

<pre>1 lang = 'python' 2 # access 3rd last element 3 print(lang[-3]) # "h"</pre>	<pre>h ></pre>
--	-------------------

3. **Slicing:** Access a range of characters in a string by using the slicing operator colon.

For example,

```
1 lang = 'python'
2 #access char from 1st index to 3rd index
3 print(lang[1:4]) # "yth"
```

String is an immutable data type, meaning that once you have created a string, you cannot change it.

```
1 lang = 'python'
2 #Trying the Modify the str
3 lang[0] = 'P'
4
5
```

ERROR!
Traceback (most recent call last):
File "<string>", line 2, in <module>
TypeError: 'str' object does not support item assignment

Strings are used widely in many different applications, such as storing and manipulating text data, representing names, addresses, and other types of data that can be represented as text.

STRINGS

Name= "Michael Jackson"

M	i	c	h	a	e	l		J	a	c	k	s	o	n
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Name[0]:M

Name[6]:l

Name[13]:o

Note: All string methods return new values. They do not change the original string.

Method

Description

[capitalize\(\)](#)

Converts the first character to upper case

<u>casefold()</u>	Converts string into lower case
<u>center()</u>	Returns a centered string
<u>count()</u>	Returns the number of times a specified value occurs in a string
<u>encode()</u>	Returns an encoded version of the string
<u>endswith()</u>	Returns true if the string ends with the specified value
<u>expandtabs()</u>	Sets the tab size of the string
<u>find()</u>	Searches the string for a specified value and returns the position of where it was found
<u>format()</u>	Formats specified values in a string
<code>format_map()</code>	Formats specified values in a string
<u>index()</u>	Searches the string for a specified value and returns the position of where it was found
<u>isalnum()</u>	Returns True if all characters in the string are alphanumeric
<u>isalpha()</u>	Returns True if all characters in the string are in the alphabet
<u>isascii()</u>	Returns True if all characters in the string are ascii characters
<u>isdecimal()</u>	Returns True if all characters in the string are decimals
<u>isdigit()</u>	Returns True if all characters in the string are digits
<u>isidentifier()</u>	Returns True if the string is an identifier

<u>islower()</u>	Returns True if all characters in the string are lower case
<u>isnumeric()</u>	Returns True if all characters in the string are numeric
<u>isprintable()</u>	Returns True if all characters in the string are printable
<u>isspace()</u>	Returns True if all characters in the string are whitespaces
<u>istitle()</u>	Returns True if the string follows the rules of a title
<u>isupper()</u>	Returns True if all characters in the string are upper case
<u>join()</u>	Joins the elements of an iterable to the end of the string
<u>ljust()</u>	Returns a left justified version of the string
<u>lower()</u>	Converts a string into lower case
<u>lstrip()</u>	Returns a left trim version of the string
<u>maketrans()</u>	Returns a translation table to be used in translations
<u>partition()</u>	Returns a tuple where the string is parted into three parts
<u>replace()</u>	Returns a string where a specified value is replaced with a specified value
<u>rfind()</u>	Searches the string for a specified value and returns the last position of where it was found
<u>rindex()</u>	Searches the string for a specified value and returns the last position of where it was found
<u>rjust()</u>	Returns a right justified version of the string

<u>rpartition()</u>	Returns a tuple where the string is parted into three parts
<u>rsplit()</u>	Splits the string at the specified separator, and returns a list
<u>rstrip()</u>	Returns a right trim version of the string
<u>split()</u>	Splits the string at the specified separator, and returns a list
<u>splitlines()</u>	Splits the string at line breaks and returns a list
<u>startswith()</u>	Returns true if the string starts with the specified value
<u>strip()</u>	Returns a trimmed version of the string
<u>swapcase()</u>	Swaps cases, lower case becomes upper case and vice versa
<u>title()</u>	Converts the first character of each word to upper case
<u>translate()</u>	Returns a translated string
<u>upper()</u>	Converts a string into upper case
<u>zfill()</u>	Fills the string with a specified number of 0 values at the beginning

String Operators

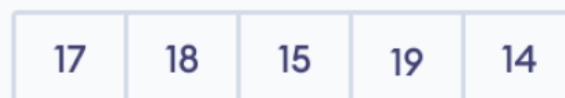
Operator	Description
+	It is known as concatenation operator used to join the strings given either side of the operator.
*	It is known as repetition operator. It concatenates the multiple copies of the same string.
[]	It is known as slice operator. It is used to access the sub-strings of a particular string.
[:]	It is known as range slice operator. It is used to access the characters from the specified range.
in	It is known as membership operator. It returns if a particular sub-string is present in the specified string.
not in	It is also a membership operator and does the exact reverse of in. It returns true if a particular substring is not present in the specified string.
r/R	It is used to specify the raw string. Raw strings are used in the cases where we need to print the actual meaning of escape characters such as "C://python". To define any string as a raw string, the character r or R is followed by the string.
%	It is used to perform string formatting. It makes use of the format specifiers used in C programming like %d or %f to map their values in python. We will discuss how formatting is done in python.

List

A list (built-in datatype/data structure) is a collection of different kinds of values or items.

Lists are just like dynamically sized arrays, declared in other languages (vector in C++ and ArrayList in Java).

Suppose we need to record the ages of **5** students. Instead of creating **5** separate variables, we can simply create a list.



List of Age

Lists Elements

Since Python lists are mutable, we can change their elements after forming. The comma (,) and the square brackets [enclose the List's items] serve as separators.

List Declaration

<pre>1 list1 = [1, 2, "Python", "Program", 15.9] 2 list2 = ["Amy", "Ryan", "Henry", "Emma"] 3 4 print(list1) 5 print(list2) 6 7 print(type(list1)) 8 print(type(list2))</pre>	<pre>[1, 2, 'Python', 'Program', 15.9] ['Amy', 'Ryan', 'Henry', 'Emma'] <class 'list'> <class 'list'> > </pre>
---	---

Accessing elements from List

<pre>1 List = ["I", "Love", "India"] 2 print("Accessing an element from the list") 3 print(List[1]) 4 print(List[2])</pre>	<pre>Accessing an element from the list Love India > </pre>
--	---

Accessing elements from 2D List

<pre>1 List = [['We', 'Love'], ['India']] 2 print("Accessing an element from 2D list") 3 print(List[0][1])</pre>	<pre>Accessing an element from 2D list Love India</pre>
--	---

The characteristics of the List are as follows:

The lists are in order.

The list element can be accessed via the index.

The mutable type of List is

The rundowns are changeable sorts.

The number of various elements can be stored in a list.

The indexing procedure is carried out similarly to string processing. The slice operator `[]` can be used to get to the List's components.

List Indexing and Slicing

The index ranges from 0 to length -1. The 0th index is where the List's first element is stored; the 1st index is where the second element is stored, and so on.

List = [0, 1, 2, 3, 4, 5]

0	1	2	3	4	5
----------	----------	----------	----------	----------	----------

List[0] = 0

List[0:] = [0,1,2,3,4,5]

List[1] = 1

List[:] = [0,1,2,3,4,5]

List[2] = 2

List[2:4] = [2, 3]

List[3] = 3

List[1:3] = [1, 2]

List[4] = 4

List[:4] = [0, 1, 2, 3]

List[5] = 5

Python lets you use negative indexing as well. The negative indices are counted from the right. The index -1 represents the final element on the List's right side, followed by the index -2 for the next member on the left, and so on, until the last element on the left is reached.

List = [0, 1, 2, 3, 4, 5]



List Methods in Python

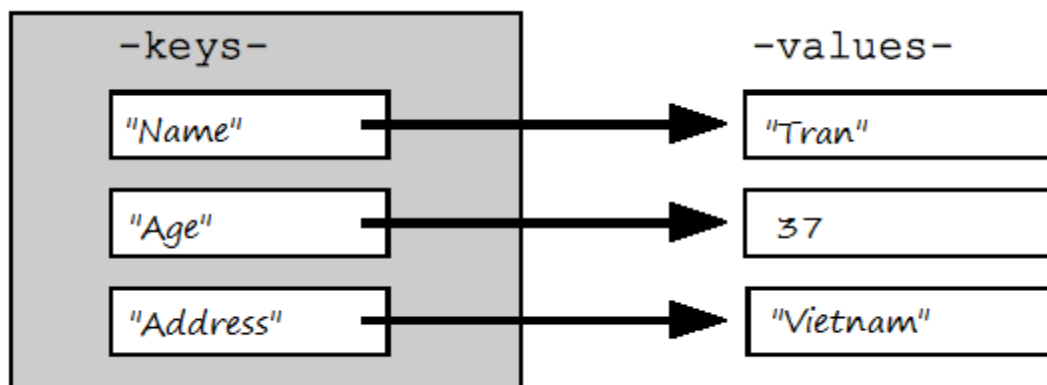
S.no	Method	Description
1	append()	Used for appending and adding elements to the end of the List.

2	<u>copy()</u>	It returns a shallow copy of a list
3	<u>clear()</u>	This method is used for removing all items from the list.
4	<u>count()</u>	These methods count the elements
5	<u>extend()</u>	Adds each element of the iterable to the end of the List
6	<u>index()</u>	Returns the lowest index where the element appears.
7	<u>insert()</u>	Inserts a given element at a given index in a list.
8	<u>pop()</u>	Removes and returns the last value from the List or the given index value.
9	<u>remove()</u>	Removes a given object from the List.
10	<u>reverse()</u>	Reverses objects of the List in place.
11	<u>sort()</u>	Sort a List in ascending, descending, or user-defined order
12	<u>min()</u>	Calculates the minimum of all the elements of the List

13	max()	Calculates the maximum of all the elements of the List
----	-----------------------	--

Dictionary

A Dictionary holds key:value pair. Key:Value is provided in the dictionary to make it more optimized.



A dictionary is a collection which is ordered*, changeable and do not allow duplicates.

As of Python version 3.7, dictionaries are *ordered*. In Python 3.6 and earlier, dictionaries are *unordered*.

Dictionaries are written with curly brackets ({}), including key-value pairs separated by commas (,).

```

1 # creating a dictionary
2 country_capitals = {
3     "United States": "Washington D.C.",
4     "Italy": "Rome",
5     "England": "London"
6 }
7 # printing the dictionary
8 print(country_capitals)

```

```

{'United States': 'Washington D.C.', 'Italy': 'Rome', 'England': 'London'}
> |

```

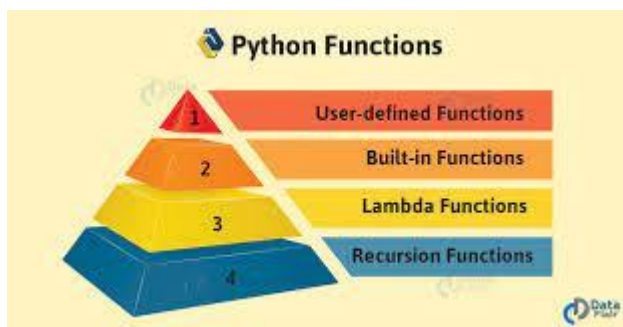
Table of Python Dictionary Methods

Functions Name	Descriptions
<u>clear()</u>	Removes all items from the dictionary
<u>copy()</u>	Returns a shallow copy of the dictionary
<u>fromkeys()</u>	Creates a dictionary from the given sequence
<u>get()</u>	Returns the value for the given key
<u>items()</u>	Return the list with all dictionary keys with values
<u>keys()</u>	Returns a view object that displays a list of all the keys in the dictionary in order of insertion
<u>pop()</u>	Returns and removes the element with the given key
<u>popitem()</u>	Returns and removes the key-value pair from the dictionary

<u>setdefault()</u>	Returns the value of a key if the key is in the dictionary else inserts the key with a value to the dictionary
<u>values()</u>	Updates the dictionary with the elements from another dictionary
<u>update()</u>	Returns a list of all the values available in a given dictionary

Functions

A collection of related assertions that carry out a mathematical, analytical, or evaluative operation is known as a function.



Functions:

A python function is a block of code or a set of statements that performs a specific task.

- The block of code is assigned with a unique identifier (name). It acts as a reference variable.
- The data is to put commonly repeatedly done tasks together and make a function . So that, instead of writing the code again and again for different inputs we can use the function call to reuse the code over and over again.
- “def” key word is used to create a function.

Advantage :

1. The main advantage of function is reusability
2. Less lines of code
3. Easy to debug or modify

Syn:

```
def function_name (parameters):    àheader/ declaration
    key word           Function signature
    #all ;pmstatements  -----à body of the function/implementation
    return expression -----àfunction return
```

Functions are classified into 2 types:

1. Predefined functions
2. User defined functions

-

Pre-defined functions:

These are some standard functions already defined in python that are available to use.

There are many modules in python that provides built in functions.

EX: len(), sum(), sorted(), reversed(), min(), max()

User-defined functions:

The functions which are defined by the user (programmer) are called user-defined functions.

We can add any type of personality or properties to a function as we required.

There are 2 steps in functions:

1. Function definition
2. Function calling

-

Function calling:

- The control goes to the function if we call the function manually by using the name of the function followed by parenthesis () containing parameters.
- The values which are passed in the function call are called actual parameters.
- The values which are assigned to variables in the function definition are called formal parameters.

Note: A function can return a data as a result.

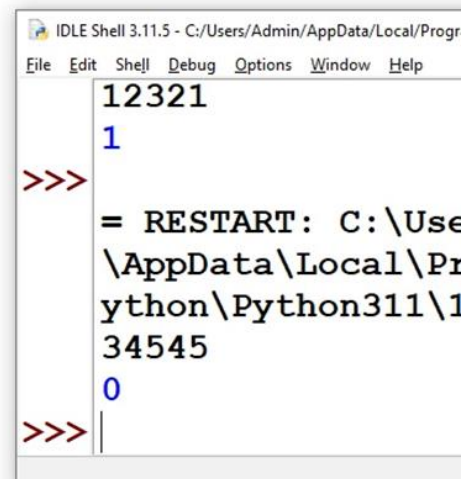
- When the function is called, the control of the program goes to the function definition.
- All codes in function are executed.
- After execution of function the control of the program jumps to the next statements after the function call.
- A function is defined once and can be called many times.

Return:

Transfer control to the called statement.

```
#WAP to print 1 if it is Palindrome, otherwise print 0
def isPalindrome(n):
    temp = n
    rev = 0
    while n>0:
        r = n%10
        rev = rev*10 + r
        n = n//10
    print(1 if rev==temp else 0)

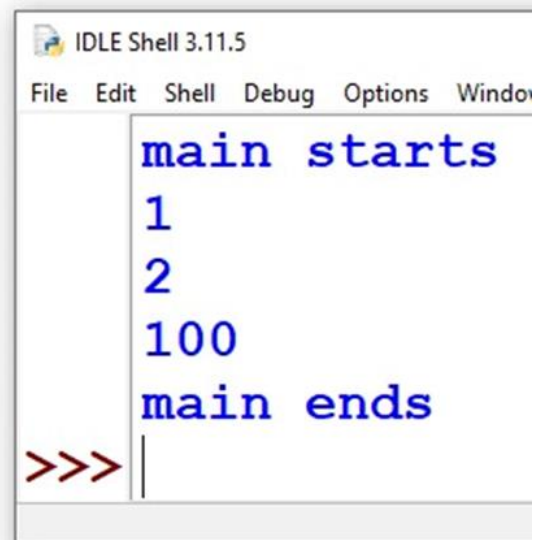
#main code
n = int(input())
isPalindrome(n)
```



```
IDLE Shell 3.11.5 - C:/Users/Admin/AppData/Local/Programs/Python/Python311/IDLE Shell
File Edit Shell Debug Options Window Help
12321
1
>>>
= RESTART: C:/Users/Admin/AppData/Local/Programs/Python/Python311/IDLE Shell
34545
0
>>>
```

```
def fun():
    print(1)
    print(2)
    return 100

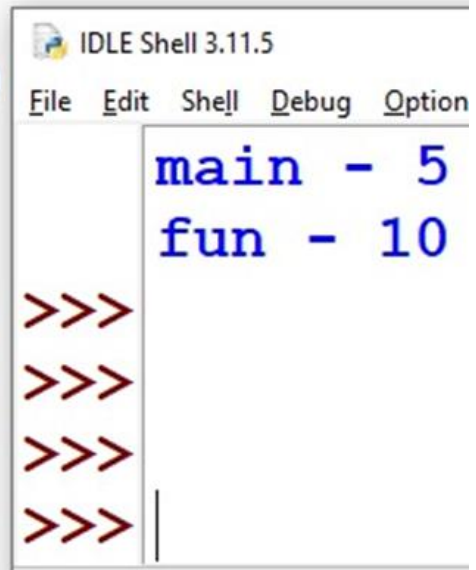
#main code
print('main starts')
z = fun()
print(z)
print('main ends')
```



The screenshot shows the IDLE Shell 3.11.5 window. The menu bar includes File, Edit, Shell, Debug, Options, and Window. The shell area displays the output of the first code block: 'main starts', '1', '2', '100', and 'main ends'. The prompt '>>>>' is visible on the left side of the shell area.

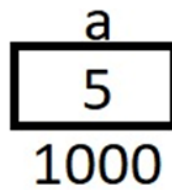
```
def fun():
    a = 10
    print('fun -', a)

#main code
a = 5
print('main -', a)
fun()
```

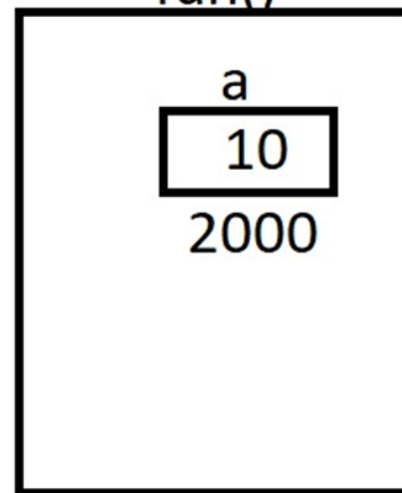


The screenshot shows the IDLE Shell 3.11.5 window. The menu bar includes File, Edit, Shell, Debug, and Option. The shell area displays the output of the second code block: 'main - 5' and 'fun - 10'. The prompt '>>>>' is visible on the left side of the shell area, appearing four times.

Main



fun()



fun@123

```
def fun():  
    print('fun -', a)
```

```
#main code
```

```
a = 5
```

```
print('main -', a)
```

```
fun()
```

```
print('main -', a)
```

IDLE Shell 3.11.5

File Edit Shell Debug Options

l\Programs

ython311\1

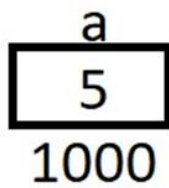
main - 5

fun - 5

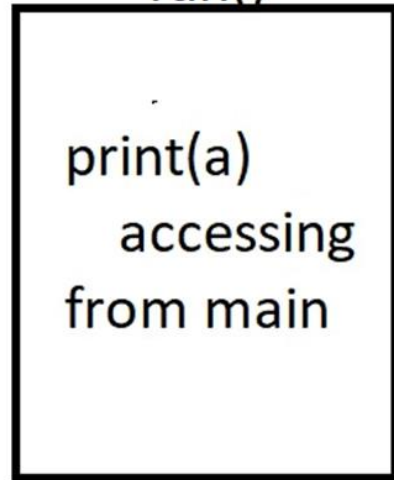
main - 5

>>>

Main



fun()



fun@123

```
def fun():  
    a = 10  
    print('fun -', a)  
  
#main code  
fun()  
print('main -', a)
```

```
fun - 10  
Traceback (most recent call last):  
  File "C:\Users\Admin\AppData\Local\Programs\Python\Python311\11am.py",  
    line 9, in <module>  
        print('main -', a)  
NameError: name 'a' is not defined
```

```
def fun():
    global a
    a = 10
    print('fun -', a)
```

#main code

```
fun()
print('main -', a)
```

>>>

```
\Admin\AppData\Local\Programs\Python\Python311\1
fun - 10
main - 10
```

main - 5

Traceback (most recent call last):

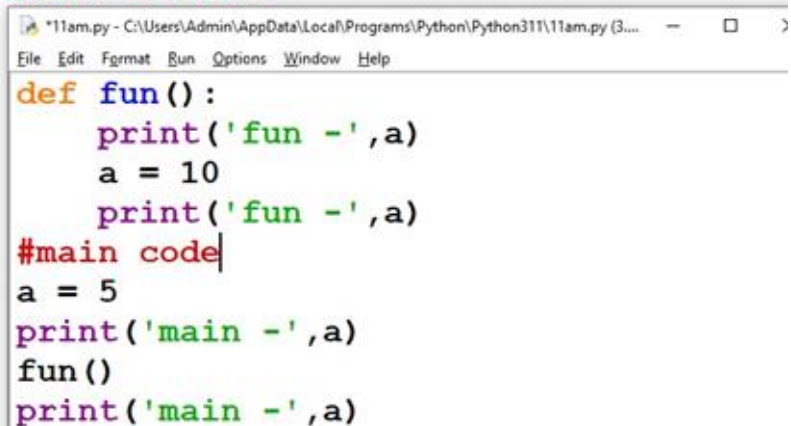
File "C:\Users\Admin\AppData\Local\Programs\Python\Python311\11am.py", line 11, in <module>

fun()

File "C:\Users\Admin\AppData\Local\Programs\Python\Python311\11am.py", line 4, in fun

print('fun -', a)

UnboundLocalError: cannot access local variable 'a' where it is not associated with a value



```
*11am.py - C:\Users\Admin\AppData\Local\Programs\Python\Python311\11am.py (3...
File Edit Format Run Options Window Help
def fun():
    print('fun -', a)
    a = 10
    print('fun -', a)
#main code
a = 5
print('main -', a)
fun()
print('main -', a)
```

Local variable:

- The variables that are declared inside the functions are called local variables. It is defined within a local scope
- We can access a local variable inside the function but not from outside

Global Variable:

The variables that are created outside the function are called global variables.

- A global variable can be accessed from any part of the program including within functions.
- If there a local variable as well as global variable with the same name
 - 1)accessing variable inside the function gives the local variable
 - 2)accessing variable outside the function gives the global variable value.
 - 3)local variables are not accessible from main code or other functions.
- We can declare a global variable inside a function by using the key word “GLOBAL”.

Programs for home work by using functions :

- 1.Even
- 2.Perfect
- 3.Palindrome
- 4.Armstrong
- 5.Strong/special
- 6.Fibonacci series

```
def isPrime(n):
    count = 0
    for i in range(1, n+1):
        if n%i==0:
            count += 1
    return count == 2
```

```
#main code
n = int(input())
a = list(map(int, input().split()))

res_a = []
for x in a:
    res = isPrime(x)
    res_a.append(res)

print(*res_a)
```

```
IDLE Shell 3.11.5
File Edit Shell Debug Options Window Help
5
2 7 1 6 3
True True False False True
>>>
>>>
```

```
def isPrime(n):
    count = 0
    for i in range(1, n+1):
        if n%i==0:
            count += 1
    return count == 2
```

```
#main code
n = int(input())
a = list(map(int, input().split()))

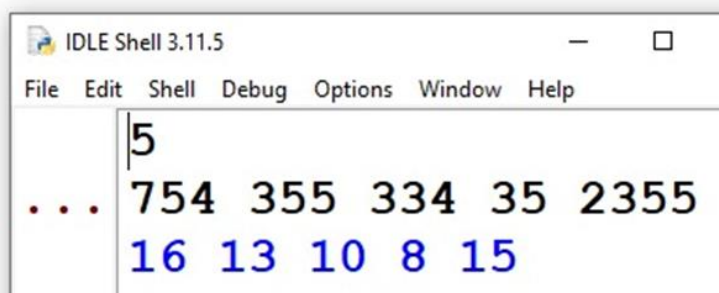
res_a = []
for x in a:
    if isPrime(x):
        res_a.append(x)

print(*res_a)
```

```
IDLE Shell 3.11.5
File Edit Shell Debug Options Window Help
10
4 1 6 5 7 11 13 14 15 28
5 7 11 13
>>>
>>>
```

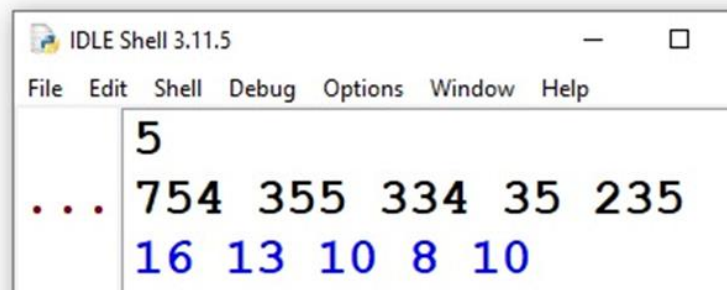
***** Sum of individual digits of every element in the list

```
def sumOfID(n):  
    #write your code here  
    sum = 0  
    while n>0: sum += n%10; n //= 10  
    return sum  
  
#main code  
n = int(input())  
a = list(map(int, input().split()))  
  
res_a = []  
for x in a:  
    res_a.append(sumOfID(x))  
  
print(*res_a)
```



```
IDLE Shell 3.11.5  
File Edit Shell Debug Options Window Help  
5  
... 754 355 334 35 2355  
16 13 10 8 15
```

```
def sumOfID(n):  
    #write your code here  
    return sum(list(map(int, str(n))))  
  
#main code  
n = int(input())  
a = input().split()  
a = list(map(int, a))  
  
res_a = list(map(sumOfID, a))  
  
print(*res_a)
```



```
IDLE Shell 3.11.5  
File Edit Shell Debug Options Window Help  
5  
... 754 355 334 35 235  
16 13 10 8 10
```

**** Printing even numbers in the given list

```
def isEven(n:int) -> bool:
    return n%2==0

#main code
n = int(input())
a = list(map(int, input().split()))
res_a = list(filter(isEven, a))
print(*res_a)
```



Map:

Map function works as an iterator and returns a result after applying the function to every element of the given iterable (sequence).

Syntax:

map(fun, iter)

parameters 1) fun--- it is the function to which map passes each element of given iterable.

2) iter: it is iterable which is to be mapped

Note: map function returns the map object as a result.

Further the map object can be converted to list, tuple etc by using list() , tuple() constructors etc.

Fibonacci series (not recommended):

```
n = int(input())
a, b = 0, 1

print(a,b,end=" ")
```

```
for i in range(n-2):
    c = a+b
    print(c, end=" ")
    a = b
    b = c
```



```
File Edit Shell Debug Options
5
0 1 1 2 3
>>>
>>>
```

Fibonacci series:

```
n = int(input())
a, b = 0, 1

for i in range(n):
    print(a, end=" ")
    c = a+b
    a = b
    b = c
```

```

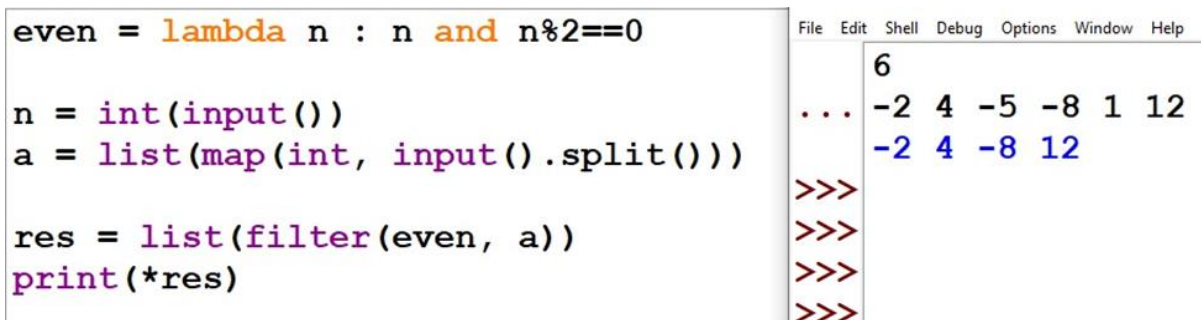
n = int(input())
a, b = 0, 1

for i in range(n):
    print(a, end=" ")
    a, b = b, a+b

```



LAMBDA:



Filter:

The filter method filters the given sequence (blocks unwanted elements and allows wanted elements) with the help of a function that tests each element in the sequence to be true or not.

Syn:

Filter (fun, iter)

Here fun means the function that tests each element of a sequence is true or not.

Iter: it is the sequence which needs to be filtered

The filter function returns an iterator that is already filtered.

Lambda:

- Lambda is a special type of function which is a anonymous (nameless) single line function.
- In functions we use def key word to define a function similarly we use lambda key word to define a lambda function.
- Lambda function can take any number of parameters (separated by comma) but can have only one expression which is evaluated and return by default.

Syn:

lambda arguments : expression

Lambda arg_1, arg_2,..... arg_n : expression

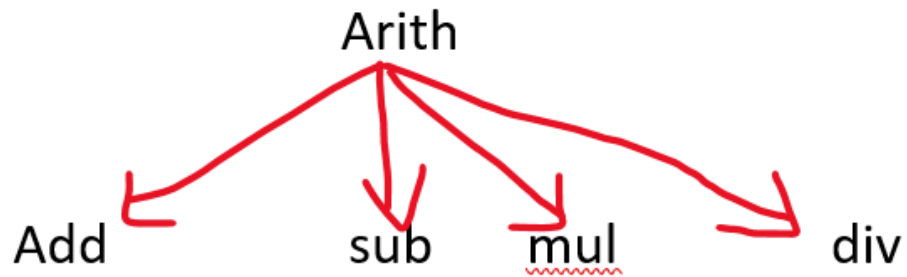
Modules:

- In python modules are simply files with “.py” extension containing python code that can be imported inside another python program
- We can consider a module to be the same as a code library or a file that contains a set of functions that you want to include in your python application.
- So It is considered a best production level projects in data science is to split large python code blocks into modules containing upto 300-400 lines of code.

Components in module:

- Definitions and implementation of classes variables.
- Function that can be used inside another program .

EX:



- The file ends with “.py” is called a module file.
- By using “import” key word we can import the module

```
import Arith
```

```
a, b = int(input()), int(input())
```

```
res_add = Arith.add(a, b)
```

```
print(res_add)
```

File Edit Shell

10

5

15

>>>

>>>

>>>

>>>

```
from Arith import add
```

```
a, b = int(input()), int(input())
```

```
res_add = add(a, b)
```

```
print(res_add)
```

File Edit Shell

5

2

7

>>>

>>>

>>>

>>>

>>>

```
from Arith import add, sub
```

```
a, b = int(input()), int(input())
```

```
res_add = add(a, b)
```

```
print(res_add)
```

```
sub = sub(a, b)
```

```
print(sub)
```

File Edit Shell

10

5

15

5

>>>

>>>

>>>

>>>

```
from Arith import *
```

```
a, b = int(input()), int(input())
```

```
res_add = add(a, b)
```

```
print(res_add)
```

```
sub = sub(a, b)
```

```
print(sub)
```

```
pro = pro(a, b)
```

```
print(pro)
```

```
div = div(a, b)
```

```
print(div)
```

```
mod = mod(a, b)
```

```
print(mod)
```

IDLE Shell 3.11.5

File Edit Shell Debug Options

5

2

7

3

10

2

1

>>>

- If we are importing total module we need to mention the module name for accessing a function or a variable or a class.

Ex: import Arith
 Res= Arith.add(4,6)
 print(Res)
 print(Arith.a)

- If we are importing particular method or by using * we need not to mention module name.

Ex: import Arith *
 Res= add(4,6)
 print(Res)
 print(a)

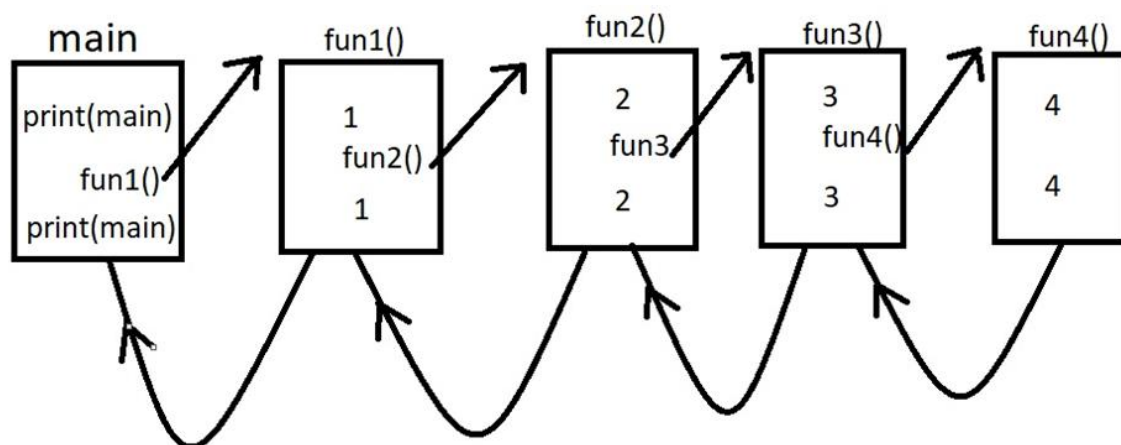
Ex: import Arith add, sub
 Res= add(4,6)
 print(Res)
 print(a)

```
Thop.py x  main.py x
1 usage
1 def fun1():
2     print(1)
3     fun2()
4     print(1)
1 usage
5 def fun2():
6     print(2)
7     fun3()
8     print(2)
1 usage
9 def fun3():
10    print(3)
11    fun4()
12    print(3)
1 usage
13 def fun4():
14    print(4)
15    print(4)

1 from pack.subPack.Thop import *
2
3 print('main')
4 fun1()
5 print('main')
6
7
8
9
```

Run main x

C:\Users
main
1
2
3
4
4
3
2
1
main



```
Thop.py x 2 usages 1 ^ v
1 def fun(n):
2     print(n)
3     fun(n-1)
4     print(n)
5
6
7
8
9
10
11
12
13
14

main.py x
1 from pack.subPack.Thop import *
2
3 print('main Start')
4 fun(5)
5 print('main End')
6
7
8
9
10
11

Run main x
C:\Users\Ad
main Start
5
4
3
2
1
0
-1
-2
-3
```

O/P: 5 4 3 2 1 0 -1 -2 -3.....

```
Thop.py x 2 usages 2 ^ v
1 def fun(n):
2     if n==0:
3         return
4     print(n)
5     fun(n-1)
6     print(n)
7
8
9
10
11
12

main.py x
1 from pack.subPack.Thop
2
3 print('main Start')
4 fun(5)
5 print('main End')
6
7
8
9
10
11

C:\Users\Adm
main Start
5
4
3
2
1
1
2
3
4
5
main End
```

```
Thop.py x  main.py x
1  #WAF to print
2  #even numbers
   2 usages
3  def fun(n):
4      if n==0:
5          return
6      fun(n-1)
7      if n%2==0:
8          print(n, end=" ")
9
10
```

```
1  from pack.subPack.Thop import *
2
3  n = int(input())
4  fun(n)
5
6
7
8
9
10
```

Run main x

C:\Users\Admin\Pyc

15

2 4 6 8 10 12 14

```
Thop.py x  main.py x
1  #WAF to print
2  #even numbers
   2 usages
3  def fun(n,i):
4      if n%i==0:
5          print(i, end=" ")
6      if n==i:
7          return
8      fun(n, i+1)
9
10
```

```
1  from pack.subPack.Thop import *
2
3  n = int(input())
4  fun(n, i: 1)
5
6
7
8
9
10
```

Run main x

C:\Users\Admin\Pyc

6

1 2 3 6

*** WAP to print factors of n in reverse order using recursion

```
def fun(n,i):
    if n+1==i:
        return
    fun(n,i+1)
    if n%i==0:
        print(i,end=" ")

n=int(input())
fun(n,1)
```

```
IDLE Shell 3.11.2
File Edit Shell Debug
>>>
= RESTART
Hema sai
rec.py
6
6 3 2 1
>>>
```

```
def fun(n,i):
    if i<n:
        fun(n,i+1)
    if n%i==0:
        print(i,end=" ")

n=int(input())
fun(n,1)
```

```
IDLE Shell 3.11.2
File Edit Shell Debug
= RESTART
Hema sai
rec.py
6
6 3 2 1
>>>
```

**** Fibonacci series using recursion

```
1 def fib(n,a=0,b=1):
2     print(a, end=" ")
3     if n>1:
4         fib(n-1,b,a+b)
5
6
7
8
9
10
```

```
1 from pack.subPack.Thop import :
2
3 n = int(input())
4 fib(n)
```

Run main x

C:\Users\Admin\PycharmProjects\p

5

0 1 1 2 3

Process finished with exit code (

**** sum of natural numbers using recursions

```
File Edit Format Run Options Window Help
def fun(n) :

    if n==1:
        return 1
    return n+fun(n-1)

n=int(input())
print(fun(n))
```

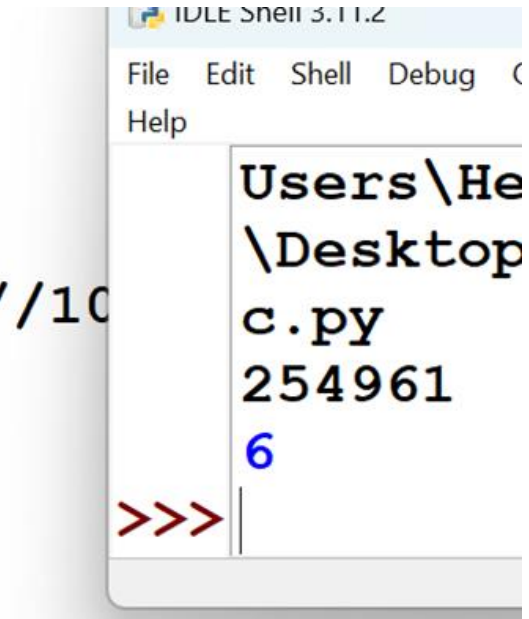
IDLE Shell 3.11.2

File Edit Shell Help

: \Us
ai \D
rec.
====
5
15

****count of digits of a given number

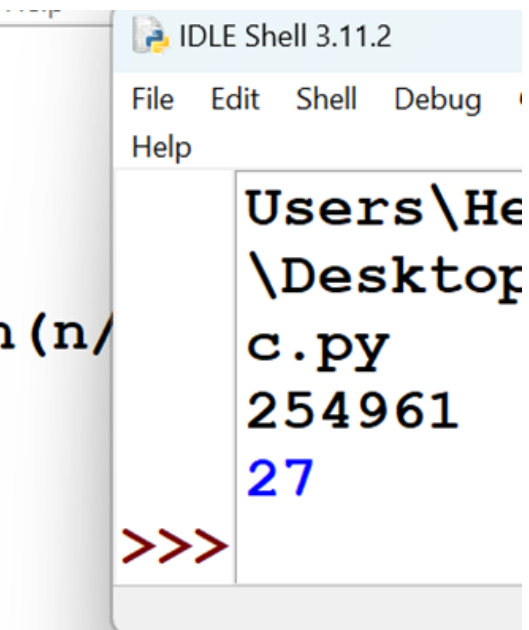
```
def fun(n):  
    if n==0:  
        return 0  
    return 1+fun(n//10)  
  
n=int(input())  
print(fun(n))
```



```
IDLE Shell 3.11.2  
File Edit Shell Debug C  
Help  
Users\He  
\Desktop  
c.py  
254961  
6  
>>>
```

***sum of digits of a given number

```
def fun(n):  
    if n==0:  
        return 0  
    return n%10+fun(n//10)  
  
n=int(input())  
print(fun(n))
```



```
IDLE Shell 3.11.2  
File Edit Shell Debug C  
Help  
Users\He  
\Desktop  
c.py  
254961  
27  
>>>
```

```
1 def fun(n):
2     if n==0:
3         return 0
4     r = n%10
5     return r + fun(n//10)
6
7
8
9
10
```

```
1 from pack.subPack.Thop import
2
3 n = int(input()) #254961
4 print(fun(n))
```

Run main x

C:\Users\Admin\PycharmProjects\py

254961

27

*** factorial of a given number

```
File Edit Format Run Options Window Help
```

```
def fun(n):
    if n==1:
        return 1
    return n*fun(n-1)

n=int(input())
print(fun(n))
```

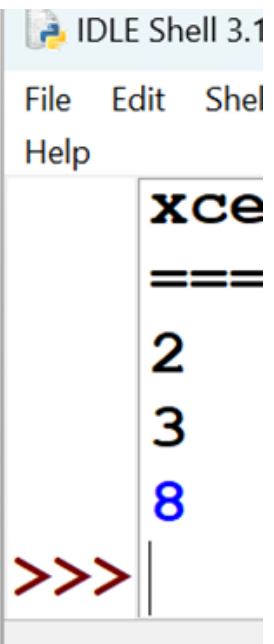
```
File Edit Shell Debug
```

```
>>>
= RESTA
ema sa:
cept.py
5
120
>>>
>>>
```

***power of any number using recursion

```
def fun(a,b):
    if b==0:
        return 1
    return a*fun(a,b-1)

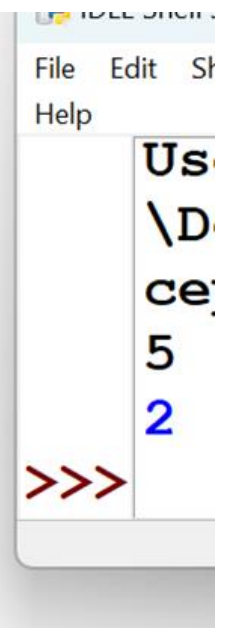
a=int(input())
b=int(input())
print(fun(a,b))
```



*****count of factors of a given number using recursion

```
def fun(n,i=1):
    if i==n:
        return 1
    if n%i==0:
        return 1+fun(n,i+1)
    return fun(n,i+1)

n=int(input())
print(fun(n))
```



```
2 usages
def fun(n,i=1):
    if i==n:
        return 1
    return(1 if n%i==0 else 0) +fun(n,i+1)

n=int(input())
print(fun(n))
```

***Sum of factors of a given number

```
def fun(n,i=1):
    if i==n:
        return n
    if n%i==0:
        return i+fun(n,i+1)
    return fun(n,i+1)
n=int(input())
print(fun(n))
```

```
2 usages
def fun(n,i=1):
    if i==n:
        return n
    return(i if n%i==0 else 0) +fun(n,i+1)

n=int(input())
print(fun(n))
```

*****Prime another logic

```
1 usage
def fun(n):
    for i in range(2,n+1):
        if n%i==0:
            break
    return(n>1 and i==n)

n=int(input())
print(fun(n))
```

"C:\Use
5
True
Process

*****Prime number using recursion and while else

```
2 usages
def fun(n,i=2):
    if i==n:
        return True

    if n==1 or n % i == 0:
        return False
    return fun(n, i + 1)

n=int(input())
print(fun(n))
```

"C:\Users
17
True
Process f
|

```
def fun(n):
    i=2
    while i<n:
        if n%i==0:
            return False
            break
        i+=1
    else:
        return True

n=int(input())
print(fun(n))
```

***perfect number using recursion

```
def fun(n,i=1):
    if i==n-1:
        return 0
    if n%i==0:
        return i+fun(n,i+1)
    return fun(n,i+1)

n=int(input())
res=fun(n)
print("yes" if res==n else "no")
```

A function calling itself is called a recursive function and this process is called recursion.

Advantages:

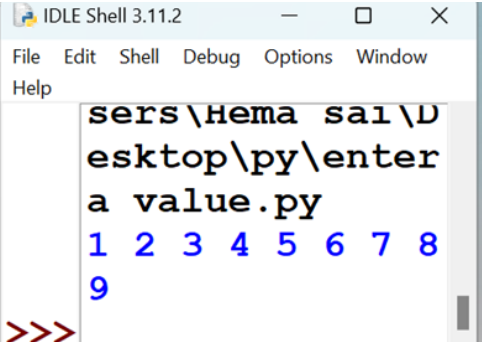
- It looks simple and effective.
- Reduces unnecessary calling of function.
- It helps you to break down the big problems into smaller ones.
- The main advantage of recursion is very useful in solving the data structure problem and it reduces the length of code.

Dis-Advantages:

- It is challenging to debug.
- Recursive functions are slower than non-recursive function.
- It is less efficient in terms of space and time complexity.

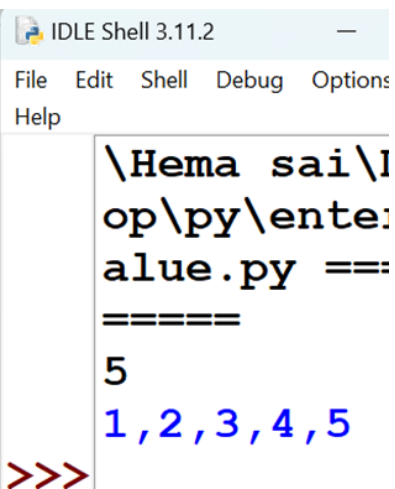
NESTED LISTS:

```
a= [[1,2,3],  
    [4,5,6],  
    [7,8,9]]  
for i in range(len(a)):  
    for j in range(len(a[i])):  
        print(a[i][j],end=" ")  
>>>
```



List comprehensions:

```
n=int(input())  
nat=[i for i in range(1,n+1)]  
print(*nat,sep=",")  
>>>
```



**without using map

```
a=list(map(fun,iter))  
a=[fun(x) for x in iter]
```

```
n=int(input())  
nat=[i for i in range(1,n+1) if i%2==0]  
print(*nat,sep=",")
```

IDLE Shell 3.11.2

File Edit Shell Debug Options Window Help

```
= RESTART: C  
sers\Hema sa  
esktop\py\en  
a value.py  
10  
2,4,6,8,10
```

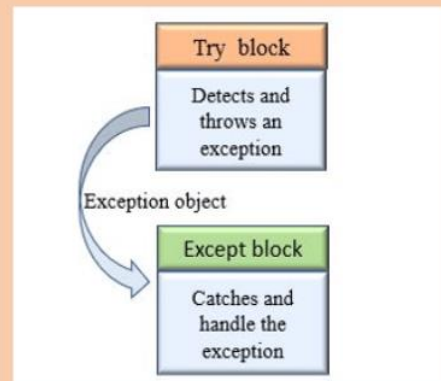
***Without using filter:

```
a=list(filter(prime,seq))  
a=[x for x in seq if prime(x)]
```

Python Exception Handling

An exception is an event that occurs during the execution of programs that **disrupt the normal flow of execution**

- 1.Try-except-finally
- 2.Raise an exception
- 3.Exception chaining
- 4.Built-in exceptions
- 5.Custom exceptions



Python Exceptions List

Here is the complete list of Python in-built exceptions.

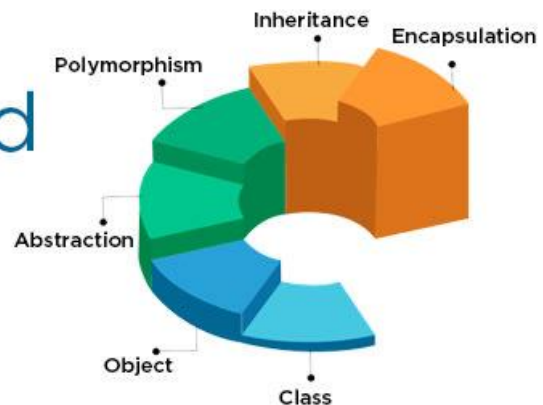
Sr.No.	Name of the Exception	Description of the Exception
1	Exception	All exceptions of Python have a base class.
2	StopIteration	If the next() method returns null for an iterator, this exception is raised.
3	SystemExit	The sys.exit() procedure raises this value.
4	StandardError	Excluding the StopIteration and SystemExit, this is the base class for all Python built-in exceptions.
5	ArithmeticError	All mathematical computation errors belong to this base class.

6	OverflowError	This exception is raised when a computation surpasses the numeric data type's maximum limit.
7	FloatingPointError	If a floating-point operation fails, this exception is raised.
8	ZeroDivisionError	For all numeric data types, its value is raised whenever a number is attempted to be divided by zero.
9	AssertionError	If the Assert statement fails, this exception is raised.
10	AttributeError	This exception is raised if a variable reference or assigning a value fails.
11	EOFError	When the endpoint of the file is approached, and the interpreter didn't get any input value by raw_input() or input() functions, this exception is raised.
12	ImportError	This exception is raised if using the import keyword to import a module fails.
13	KeyboardInterrupt	If the user interrupts the execution of a program, generally by hitting Ctrl+C, this exception is raised.
14	LookupError	LookupErrorBase is the base class for all search errors.
15	IndexError	This exception is raised when the index attempted to be accessed is not found.

16	KeyError	When the given key is not found in the dictionary to be found in, this exception is raised.
17	NameError	This exception is raised when a variable isn't located in either local or global namespace.
18	UnboundLocalError	This exception is raised when we try to access a local variable inside a function, and the variable has not been assigned any value.
19	EnvironmentError	All exceptions that arise beyond the Python environment have this base class.
20	IOError	If an input or output action fails, like when using the print command or the open() function to access a file that does not exist, this exception is raised.
22	SyntaxError	This exception is raised whenever a syntax error occurs in our program.
23	IndentationError	This exception was raised when we made an improper indentation.
24	SystemExit	This exception is raised when the sys.exit() method is used to terminate the Python interpreter. The parser exits if the situation is not addressed within the code.
25	TypeError	This exception is raised whenever a data type-incompatible action or function is tried to be executed.

26	ValueError	This exception is raised if the parameters for a built-in method for a particular data type are of the correct type but have been given the wrong values.
27	RuntimeError	This exception is raised when an error that occurred during the program's execution cannot be classified.
28	NotImplementedError	If an abstract function that the user must define in an inherited class is not defined, this exception is raised.

Object Oriented Programming with Python



Constructor

- A constructor is a special type of method that gets invoked (called) automatically when an instance (object) is created of a class.
- The main purpose of a constructor is to initialize or assign values to the data members of that class. It cannot return any value other than none.

- Constructors in C++ or Java have the same name as the class, but Python considers constructors.
- In Python, a constructor is called by the `__init__()` method, and it is invoked whenever an object is created.
- On the other hand, a destructor is used to destroy the object.

Note: In Python, internally, the `__new__` is the method that creates the object, and `__del__` method is called to destroy the object when the reference count for that object becomes zero.

- In Python, Object creation is divided into two parts in Object Creation and Object initialization
- Internally, the `__new__` is the method that creates the object.
- And, using the `__init__()` method we can implement constructor to initialize the object.

Syntax

```
class Sample():
```

```
def __init__(self)::
```

```
# body of the constructor
```

Where,

def: The keyword is used to define function.

`__init__()` Method: It is a reserved method. This method gets called as soon as an object of a class is instantiated.

self: The first argument self refers to the current object. It binds the instance to the `__init__()` method. It's usually named self to follow the naming convention.

Note: The `__init__()` method arguments are optional. We can define a constructor with any number of arguments.

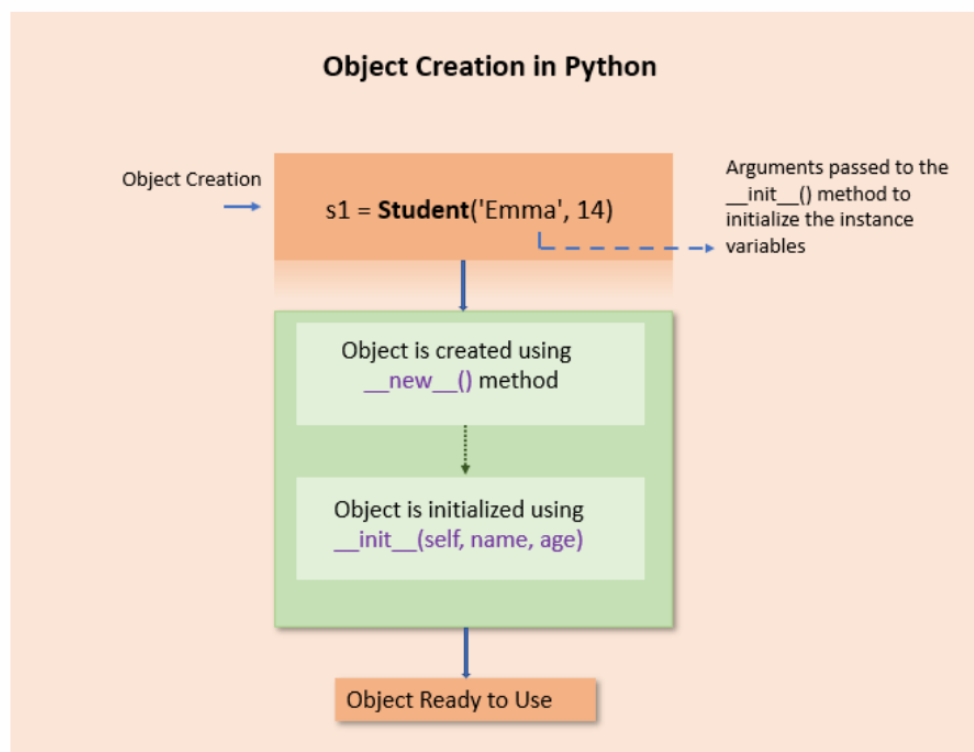
Example

We'll create a Class Trainer with an instance variable trainer name. we'll see how to use a constructor to initialize the trainer name at the time of object creation.

```
1 class Trainer:
2     def __init__(self, name):
3         print('Inside Constructor')
4         self.name = name
5         print('All variables initialized')
6
7     # instance Method
8     def show(self):
9         print('Hello, my name is', self.name)
10
11 # create object using constructor
12 s1 = Trainer('Akhil')
13 s1.show()
14
```

Inside Constructor
All variables initialized
Hello, my name is Akhil
>

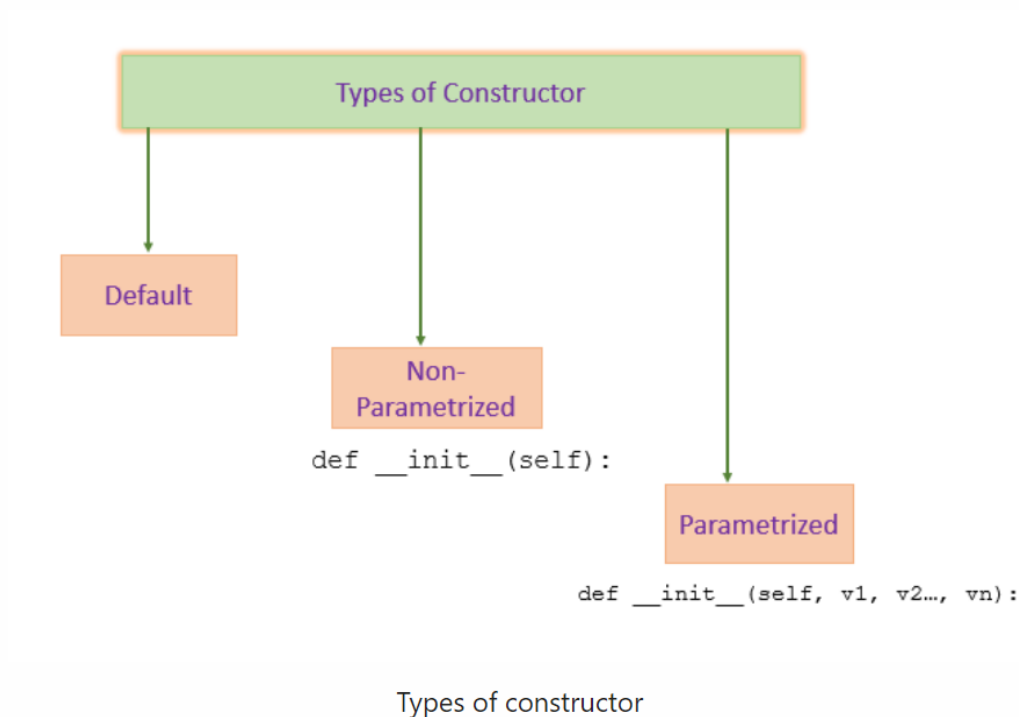
- In the above example, an object s1 is created using the constructor
- While creating a Trainer object name is passed as an argument to the `__init__()` method to initialize the object.
- Similarly, various objects of the Trainer class can be created by passing different names as arguments.



Create an object in Python using a constructor

Note:

- For every object, the constructor will be executed only once. For example, if we create four objects, the constructor is called four times.
- In Python, every class has a constructor, but it's not required to define it explicitly. Defining constructors in class is optional.
- Python will provide a default constructor if no constructor is defined.



Self keyword :

- The class contains instance variables and methods. Whenever we define instance methods for a class, we use **self** as the first parameter. Using **self**, we can access the *instance variable* and *instance method* of the object.
- The first argument **self** refers to the current object.
- Whenever we call an instance method through an object, the Python compiler implicitly passes object reference as the first argument commonly known as **self**.

- It is not mandatory to name the first parameter as a self. We can give any name whatever we like, but it has to be the first parameter of an instance method.

Instance method: Used to access or modify the object state. If we use instance variables inside a method, such methods are called instance methods. It must have a self parameter to refer to the current object.

Class method: Used to access or modify the class state. In method implementation, if we use only class variables, then such type of methods we should declare as a class method. The class method has a cls parameter which refers to the class.

Static method: It is a general utility method that performs a task in isolation. Inside this method, we don't use instance or class variable because this static method doesn't take any parameters like self and cls.

CLASS METHOD:

- Class methods are methods that are called on the class itself, not on a specific object instance. Therefore, it belongs to a class level, and all class instances share a class method.
- A class method is bound to the class and not the object of the class. It can access only class variables.
- It can modify the class state by changing the value of a class variable that would apply across all the class objects.
- In method implementation, if we use only class variables, we should declare such methods as class methods. The class method has a cls as the first parameter, which refers to the class.
- Class methods are used when we are dealing with factory methods. Factory methods are those methods that return a class object for different use cases. Thus, factory methods create concrete implementations of a common interface.

- The class method can be called using `ClassName.method_name()` as well as by using an object of the class.

Syntax for Class Method in Python

- The classmethod function receives a function as the only argument. The first parameter of the function has to be the class/instance.
- **classmethod(function_name)**

Parameters for Class Method in Python

- `classmethod()` accepts a single parameter, function that is to be converted into classmethod.

<pre> 1 class thops: 2 course = 'Python Full Stack' 3 4 def purchase(obj): 5 print("Purchase course : ", obj.course) 6 7 8 thops.purchase = classmethod(thops.purchase) 9 thops.purchase() </pre>	<pre> Purchase course : Python Full Stack > </pre>
<pre> 1 class Student: 2 # create a variable 3 name = "ThopsTech" 4 # create a function 5 def print_name(obj): 6 print("The name is : ", obj.name) 7 8 # create print_name classmethod 9 # before creating this line print_name() 10 # It can be called only with object not with class 11 Student.print_name = classmethod(Student.print_name) 12 13 # now this method can be called as classmethod 14 # print_name() method is called a class method 15 Student.print_name() </pre>	<pre> The name is : ThopsTech > </pre>

- We can use the `@classmethod` decorator to achieve the same results.

class C:

- **@classmethod**

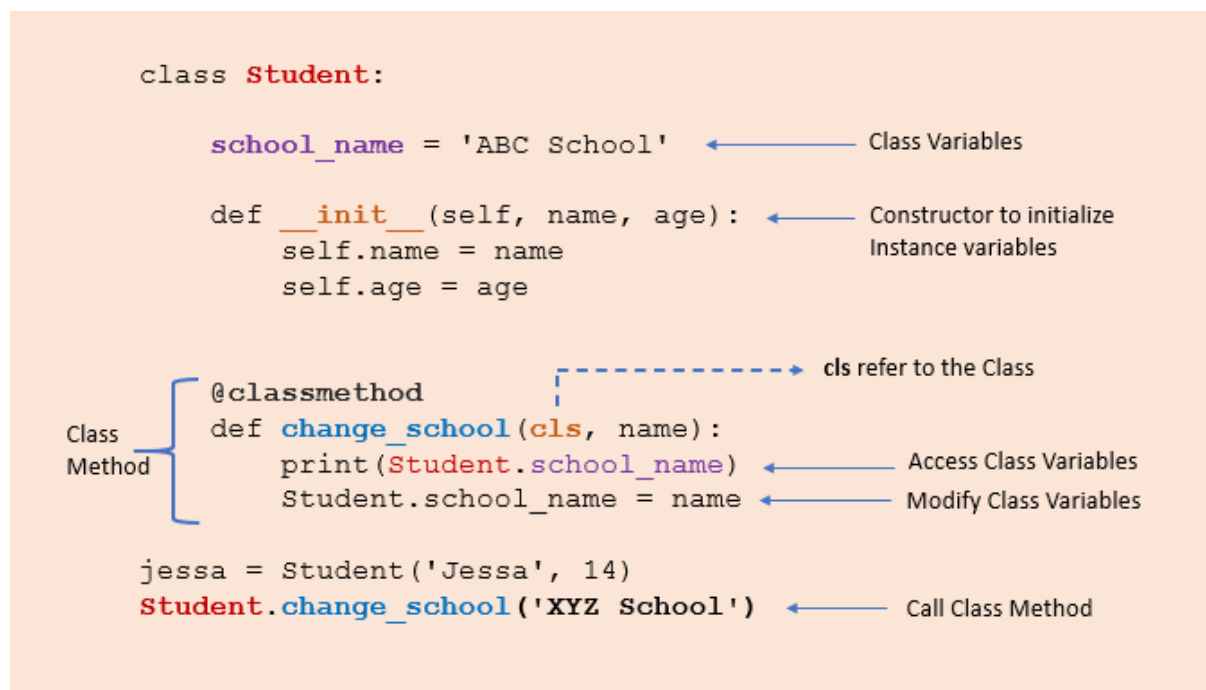
- **def f(cls, arg1, arg2): ...**
- **# Here, class is passed via "cls".**

Where,

fun: the function that needs to be converted into a class method

returns: a class method for function.

Note: Class variables are not to be confused with instance variables.



- Any method we create in a class will automatically be created as an instance method. We must explicitly tell Python that it is a class method using the `@classmethod` decorator or `classmethod()` function.
- The `@classmethod` decorator is a built-in function decorator.

Class method vs Static method

- A class method takes `cls` as the first parameter while a static method needs no specific parameters.
- A class method can access or modify the class state while a static method can't access or modify it.

- In general, static methods know nothing about the class state. They are utility-type methods that take some parameters and work upon those parameters. On the other hand class methods must have class as a parameter.
- We use `@classmethod` decorator in python to create a class method and we use `@staticmethod` decorator to create a static method in python.

Static method

- A static method does not receive an implicit first argument.
- A static method is also a method that is bound to the class and not the object of the class.
- This method can't access or modify the class state.
- It is present in a class because it makes sense for the method to be present in class.

Python Static methods can be created in two ways.

Using `staticmethod()`

```
1 class Calculator:
2
3     def add_numbers(num1, num2):
4         return num1 + num2
5
6     # convert add_numbers() to static method
7     Calculator.add_numbers = staticmethod(Calculator
8                                           .add_numbers)
9
10    sum = Calculator.add_numbers(5, 7)
11    print('Sum:', sum)
```

Sum: 12

Using `@staticmethod`

<pre> 1 class Calculator: 2 3 # create addNumbers static method 4 @staticmethod 5 def addNumbers(x, y): 6 return x + y 7 8 print('Product:', Calculator.addNumbers(15, 110)) </pre>	<pre> Product: 125 > </pre>
---	----------------------------------

ACCESS MODIFIERS

Access specifiers or access modifiers in python programming are used to limit the access of class variables and class methods outside of class while implementing the concepts of inheritance. This can be achieved by: Public, Private and Protected keyword.

Public Access Modifier

The members of a class that are declared public are easily accessible from any part of the program. All data members and member functions of a class are public by default.

<pre> 1 class Student: 2 schoolName = 'XYZ School' # class attribute 3 4 def __init__(self, name, age): 5 self.name=name # instance attribute 6 self.age=age # instance attribute 7 8 std = Student("Steve", 25) 9 print(std.schoolName) 10 print(std.name) 11 std.age = 20 12 print(std.age) </pre>	<pre> ^ XYZ School Steve 20 > </pre>
---	---

Protected Access Modifier

The members of a class that are declared protected are only accessible to a class derived from it. Data members of a class are declared protected by adding a single underscore ‘_’ symbol before the data member of that class.

```
1 class Student:
2     _schoolName = 'XYZ School' # protected class attribute
3
4     def __init__(self, name, age):
5         self._name=name # protected instance attribute
6         self._age=age # protected instance attribute
7
8 std = Student("Akhil", 25)
9 print(std._name)
10 std._name = 'Tyson'
11 print(std._name)
```

Akhil
Tyson
>

protected members of a class can be accessed by other members within the class and are also available to their subclasses. No other entity can access these members. In order to do so, they can inherit the parent class.

Private Access Modifier

The members of a class that are declared private are accessible within the class only, private access modifier is the most secure access modifier. Data members of a class are declared private by adding a double underscore ‘__’ symbol before the data member of that class.

```
class Student:
    __schoolName = 'XYZ School' # private class attribute

    def __init__(self, name, age):
        self.__name=name # private instance attribute
        self.__salary=age # private instance attribute
    def __display(self): # private method
        print('This is private method.')

std = Student("Bill", 25)
print(std.__schoolName)
#AttributeError: 'Student' object has no attribute '__schoolName'
print(std.__name)
#AttributeError: 'Student' object has no attribute '__name'
std.__display()
#AttributeError: 'Student' object has no attribute '__display'
```

Python performs *name mangling* of private variables. Every member with a double underscore will be changed to `_object._class__variable`. So, it can still be accessed from outside the class, *but the practice should be refrained*.

```
class Student:
    __schoolName = 'XYZ School' # private class attribute

    def __init__(self, name, age):
        self.__name=name # private instance attribute
        self.__salary=age # private instance attribute
    def __display(self): # private method
        print('This is private method.')

std = Student("Bill", 25)
print(std._Student__schoolName)
#AttributeError: 'Student' object has no attribute '__schoolName'
print(std._Student__name)
#AttributeError: 'Student' object has no attribute '__name'
std._Student__display()
#AttributeError: 'Student' object has no attribute '__display'
```

The reason for this is one of the main points in Python's design philosophy :

“WE ARE ALL CONSENTING ADULTS HERE”

DATA HIDING

- Data hiding is an object-oriented programming (OOP) technique specifically used to hide internal object details (i.e., data members).
- It maintained the data integrity and restricted access to the class member.
- The main working of data hiding is that it combines the data and functions into a single unit to conceal data within a class. We cannot directly access the data from outside the class.
- Technically, Data hiding isolates the client from a part of program implementation. Some of the essential members must be hidden from the user.
- Programs or modules only reflected how we could use them, but users cannot be familiar with how the application works. Thus it provides *security* and avoiding dependency as well.

Advantages of Data Hiding

- The class objects are disconnected from the irrelevant data.
- It enhances the security against hackers that are unable to access important data.
- It isolates object as the basic concept of OOP.
- It helps programmer from incorrect linking to the corrupt data.
- We can isolate the object from the basic concept of OOP.
- It provides the high security which stops damage to violate data by hiding it from the public.

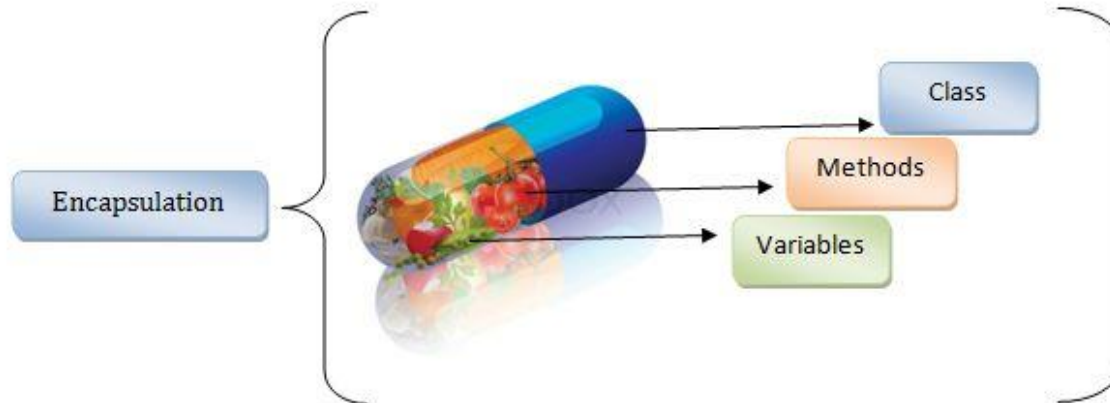
Disadvantages of Data Hiding

- It forces the programmers to write extra code to hide the important data from the common users.
- The data hiding prevents linkage that act as link between visible and invisible data makes the object faster.

but these are avoidable in front of its advantages.

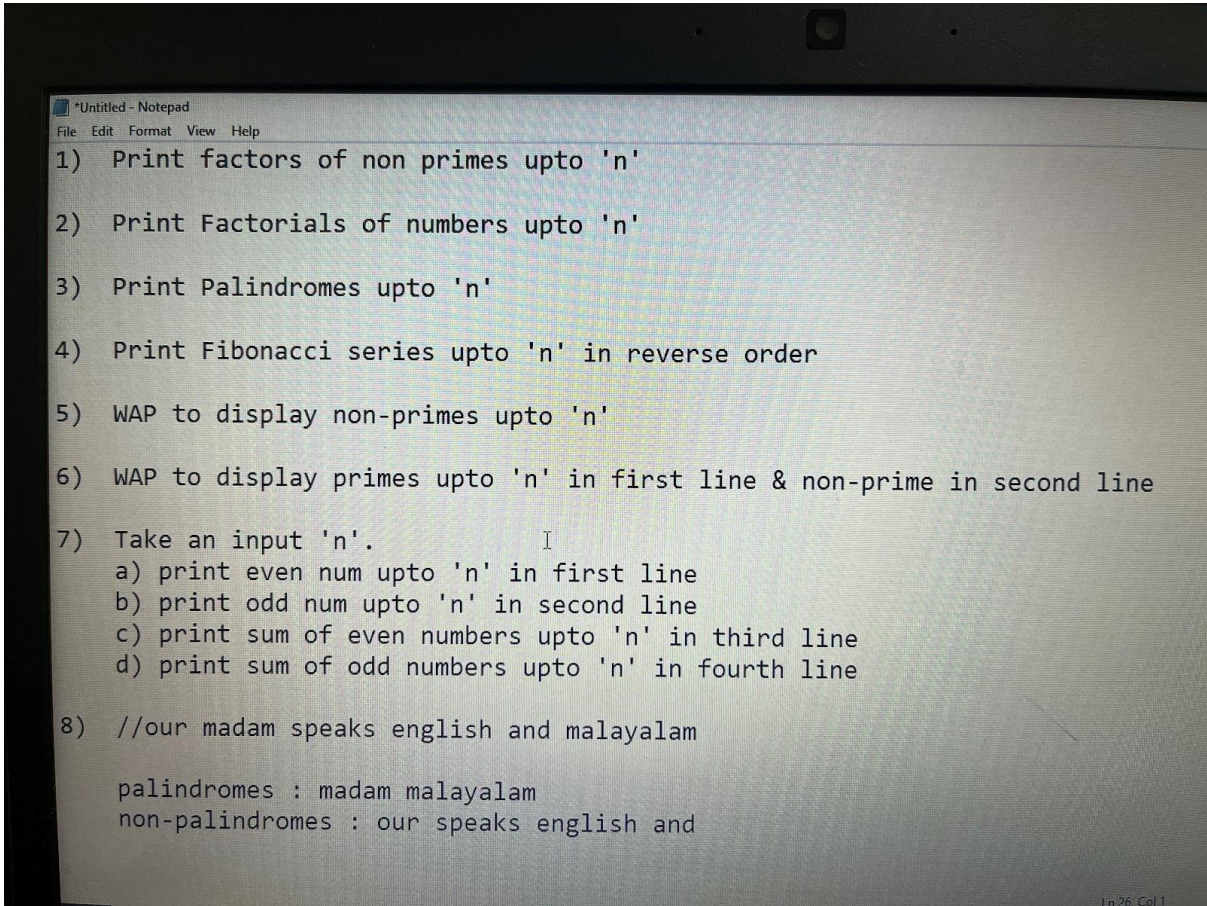
ENCAPSULATION

- Encapsulation is a mechanism of wrapping the data (private variables) and code acting on the data (public methods) together as a single unit.
- A class is an example of encapsulation as it binds all the data members (instance variables) and methods into a single unit.
- We use protected & private modifier here, this puts restrictions on accessing variables and methods directly and can *prevent the accidental modification of data*.
- To prevent accidental change, an object's variable can only be changed by an object's method.



```
Acc.py x main.py
2 usages
1 class Account:
2     bankName = "SBI"
3     def __init__(self, accName, accNo, accType, accBal):
4         self.__accName = accName #private
5         self.__accNo = accNo #private
6         self.__accType = accType #private
7         self.__accBal = accBal #private
8
9     def withdraw(self, amount):
10        self.__accBal -= amount
11        print(amount, 'rs withdrawn successfully')
12        print(self.getaccBal())
13
14    1 usage
15    def deposit(self, amount):
16        self.__accBal += amount
17        print(amount, 'rs deposited successfully')
18        print(self.getaccBal())
19
20    3 usages
21    def getaccBal(self):
22        return self.__accBal
23
24    1 usage
25    def getaccName(self):
26        return self.__accName
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
263
```

INHERITANCE



The screenshot shows a Notepad window titled '*Untitled - Notepad' with a menu bar (File, Edit, Format, View, Help). It contains a list of 8 tasks:

- 1) Print factors of non primes upto 'n'
- 2) Print Factorials of numbers upto 'n'
- 3) Print Palindromes upto 'n'
- 4) Print Fibonacci series upto 'n' in reverse order
- 5) WAP to display non-primes upto 'n'
- 6) WAP to display primes upto 'n' in first line & non-prime in second line
- 7) Take an input 'n'.
 - a) print even num upto 'n' in first line
 - b) print odd num upto 'n' in second line
 - c) print sum of even numbers upto 'n' in third line
 - d) print sum of odd numbers upto 'n' in fourth line
- 8) //our madam speaks english and malayalam

palindromes : madam malayalam
non-palindromes : our speaks english and

- Inheritance in object-oriented programming (OOP) allows you to create a new class by inheriting properties and methods from an existing class. The new class is called a subclass or derived class, and the existing class is called the superclass or base class.

In Python, inheritance is implemented using the following syntax:



```
python
class BaseClass:
    # Base class code

class DerivedClass(BaseClass):
    # Derived class code
```

Here's a simple example to demonstrate inheritance in Python:

```
python Copy  
  
class Animal:  
    def __init__(self, name):  
        self.name = name  
  
    def speak(self):  
        return "I am an animal."  
  
class Dog(Animal):  
    def speak(self):  
        return "Woof! Woof!"  
  
class Cat(Animal):  
    def speak(self):  
        return "Meow! Meow!"  
  
animal = Animal("Generic Animal")  
dog = Dog("Buddy")  
cat = Cat("Whiskers")  
  
print(animal.speak()) # Output: I am an animal.  
print(dog.speak()) # Output: Woof! Woof!  
print(cat.speak()) # Output: Meow! Meow!
```

In this example, Dog and Cat classes inherit from the Animal class. The speak method is overridden in both Dog and Cat classes to provide their own implementation.